

文件名: run_example.sh

功能: 启动配置检查与完整处理流程的命令行运行入口。

```
#!/usr/bin/env bash
set -euo pipefail
5 cd -- "$(dirname -- "${BASH_SOURCE[0]}")"
  exec "${EGOSMPLX_CONTROLLER_PYTHON:-python3}" -m egosmplx_pipeline "${1:-check}" --con
  fig "${EGOSMPLX_CONFIG_FILE:-configs/pipeline.example.json}" "${@:2}"
```

文件名: egosmplx_pipeline/__init__.py

功能: 自动鱼眼人体拟合与固定规则手部网格融合软件包。

```
10 """EgoSmplx: automatic fisheye body fitting and fixed-rule MANO fusion."""
   __version__ = "1.0.0"
```

文件名: egosmplx_pipeline/__main__.py

功能: 提供配置检查、处理运行和视频抽帧的命令行接口。

```
"""Command-line interface: check, run, sample."""
15 import argparse
   from pathlib import Path
   from egosmplx_pipeline.configuration import load_config, preflight
   from egosmplx_pipeline.io import write
   def main():
20     parser = argparse.ArgumentParser(description='EgoSmplx V1.0 automatic fisheye pose pipeline')
       sub = parser.add_subparsers(dest='command', required=True)
       for name in ['check', 'run']:
           cmd = sub.add_parser(name)
           cmd.add_argument('--config', type=Path, required=True)
25     cmd.add_argument('--cached-predictions', action='store_true')
           if name == 'run':
               cmd.add_argument('--resume', action='store_true')
           sample = sub.add_parser('sample')
           sample.add_argument('--video', type=Path, required=True)
30     sample.add_argument('--calibration', type=Path, required=True)
           sample.add_argument('--camera', required=True)
           sample.add_argument('--count', type=int, default=4)
           sample.add_argument('--output', type=Path, required=True)
           args = parser.parse_args()
35     if args.command == 'check':
           config, rows = load_config(args.config)
           print(preflight(config, rows, args.cached_predictions))
       elif args.command == 'run':
           from egosmplx_pipeline.pipeline import run
40     print(run(args.config, args.cached_predictions, args.resume))
       else:
           from egosmplx_pipeline.sampling import sample_video
           print(sample_video(args.video, args.calibration, args.camera, args.count, args.output))
       if __name__ == '__main__':
45     main()
文件名: egosmplx_pipeline/configuration.py
功能: 解析可移植配置, 并在运行前检查输入有效性。
"""Portable JSON configuration and fail-fast input checks."""
import os
50 import re
```

```

from pathlib import Path
from egosmplx_pipeline.io import read, sha
from egosmplx_pipeline.profiles import selected
PACKAGE_ROOT = Path(__file__).resolve().parents[1]
5 def expand(value, base):
    if isinstance(value, dict):
        return {k: expand(v, base) for k, v in value.items()}
    if isinstance(value, list):
        return [expand(v, base) for v in value]
10 if isinstance(value, str):
    value = value.replace('${PACKAGE_ROOT}', str(PACKAGE_ROOT))
    value = os.path.expandvars(value)
    if re.search(r'\${[^}]+\}', value):
        raise ValueError('Unresolved environment variable: ' + value)
15 return value
def absolute(path, base):
    p = Path(path).expanduser()
    return str((base / p).resolve() if not p.is_absolute() else p.resolve())
def load_config(path):
20 path = Path(path).resolve()
    config = expand(read(path), path.parent)
    config['pipeline_profile'] = selected(config)
    for key, value in config['paths'].items():
        if value:
25 config['paths'][key] = absolute(value, path.parent)
    config['manifest'] = absolute(config['manifest'], path.parent)
    config['output'] = absolute(config['output'], path.parent)
    fit = config.setdefault('fitting', {})
    for key, default in [(('rigid_steps', 1800), ('pose_steps', 1600), ('mano_guided_steps', 1400))]:
30 fit.setdefault(key, default)
    if 'silhouette_weight' in fit:
        raise ValueError('The reference profile applies contours after wrist refinement; remove fitting.silhouette_weight')
    if any(not isinstance(fit[k], int) or isinstance(fit[k], bool) or fit[k] < 1 for k in ['rigid_steps', 'pose_steps', 'mano_guided_steps']):
35 raise ValueError('Invalid fitting settings')
    manifest = read(config['manifest'])
    frames = expand(manifest['frames'], Path(config['manifest']).parent)
    ids = set()
40 for row in frames:
    identifier = row['id']
    if not re.fullmatch(r'[A-Za-z0-9_-]+', identifier) or identifier in ids:
        raise ValueError('Frame IDs must be unique safe filenames: ' + identifier)
    ids.add(identifier)
45 for key in ['image', 'calibration']:
    row[key] = absolute(row[key], Path(config['manifest']).parent)
    for key, value in row.get('predictions', {}).items():
        row['predictions'][key] = absolute(value, Path(config['manifest']).parent)
    if not frames:
50 raise ValueError('Empty input manifest')

```

```

    return config, frames
def preflight(config, frames, cached):
    """Check required files and image/observation provenance before any output."""
    import cv2
5   required = ['body_python', 'egosmplx_repo', 'egosmplx_config']
    if not cached:
        required += ['pose_python', 'wilor_python', 'sapiens_repo', 'wilor_repo',
                    'egosmplx_checkpoint', 'pose_checkpoint', 'seg_checkpoint',
                    'wilor_checkpoint', 'wilor_config', 'wilor_detector']
10   for key in required:
        if key not in config['paths'] or not Path(config['paths'][key]).exists():
            raise FileNotFoundError('Required configured path: ' + key)
    for row in frames:
        image = cv2.imread(row['image'])
15   if image is None:
        raise ValueError('Unreadable image: ' + row['image'])
    if image.shape[1::-1] != (1280, 720):
        raise ValueError('Reference profile requires 1280 x 720 images: ' + row['id'])
    calibration = read(row['calibration'])
20   if calibration['size'] != list(image.shape[1::-1]):
        raise ValueError('Image and calibration sizes differ: ' + row['id'])
    if cached:
        for key in ['egosmplx', 'sapiens', 'segmentation', 'wilor']:
            if not Path(row.get('predictions', {}).get(key, "")).exists() or key not in row.get('predictions', {}):
25   raise FileNotFoundError(row['id'] + ': missing cached prediction ' + key)
        points = read(row['predictions']['sapiens'])
        if points['image_sha256'] != sha(row['image']):
            raise ValueError('Stale Sapiens2 observation: ' + row['id'])
        for name in ['labels.npy', 'probabilities.npz']:
30   if not (Path(row['predictions']['segmentation']) / name).is_file():
            raise FileNotFoundError('Incomplete segmentation: ' + row['id'])
    if not cached:
        for name in ['pose', 'seg']:
            expected = config.get('checkpoint_sha256', {}).get(name)
35   if expected is None or not re.fullmatch('[a-f0-9]{64}', expected):
            raise ValueError('Configure checkpoint_sha256.' + name)
    return dict(passed=True, frame_count=len(frames), cached_predictions=cached,
               scope='paths, input sizes, cache image hashes; models are loaded strictly by workers')
文件名: egosmplx_pipeline/pipeline.py
40 功能: 顺序调度多环境任务, 核验输入摘要并验证续跑条件。
    """Sequential multi-environment controller with input hashing and verified resume."""
    import copy
    import datetime
    import fcntl
45   import hashlib
    import os
    from pathlib import Path
    import shutil
    import subprocess
50   import sys

```

```

from egosmplx_pipeline.configuration import PACKAGE_ROOT, load_config, preflight
from egosmplx_pipeline.io import read, write, sha
from egosmplx_pipeline.profiles import selected
def command(args, log, cwd, environment):
5     print('RUN', log.name, flush=True)
    with log.open('w') as stream:
        process = subprocess.run(list(map(str, args)), cwd=str(cwd), env=environment,
                                stdout=stream, stderr=subprocess.STDOUT)
    if process.returncode:
10         raise RuntimeError('Worker failed; inspect ' + str(log))
def environment(config):
    result = os.environ.copy()
    result.update(OMP_NUM_THREADS='2', MKL_NUM_THREADS='2', PYTHONDONTWRITEBYT
ECODE='1',
15         CUDA_VISIBLE_DEVICES=str(config.get('gpu', '0')),
        EGOSMPLX_CONFIG=config['paths']['egosmplx_config'],
        EGOSMPLX_REPO_ROOT=config['paths']['egosmplx_repo'])
    result['PYTHONPATH'] = os.pathsep.join([str(PACKAGE_ROOT), config['paths'].get('wilor_repo', "")])
    return result
20 def observe(config, rows, output, env):
    """Invoke independent external networks. No manual annotation input."""
    p = config['paths']
    observations = output / 'observations'
    observations.mkdir(exist_ok=False)
25     staged = observations / 'images'
    staged.mkdir()
    observed_rows = copy.deepcopy(rows)
    for row in observed_rows:
        image = Path(row['image'])
30         target = staged / (row['id'] + image.suffix.lower())
        shutil.copy2(image, target)
        row['image'] = str(target)
    manifest = observations / 'manifest.json'
    write(manifest, dict(frames=observed_rows))
35     for row in observed_rows:
        directory = observations / row['id']
        directory.mkdir()
        command([p['body_python'], '-m', 'egosmplx_pipeline.egosmplx_predict', '--image-dir', staged, '--ima
ge-glob', Path(row['image']).name,
40         '--fisheye-calibration', row['calibration'], '--config-path', p['egosmplx_config'],
        '--checkpoint-path', p['egosmplx_checkpoint'], '--output-dir', directory / 'network',
        '--smplx-input-mode', 'det_undistort_contiguous', '--contiguous-roii-mode', 'aspect_pad'],
        directory / 'network.log', p['egosmplx_repo'], env)
    candidates = sorted((directory / 'network').rglob(row['id'] + '*_smplx.npz'))
45     # Some backend versions return the timestamp directory beside the request.
    if not candidates:
        candidates = sorted(directory.glob('network*/smplx/' + row['id'] + '*_smplx.npz'))
    if len(candidates) != 1:
        raise RuntimeError('Expected one wearer prediction, found ' + str(len(candidates)))
50     args = [p['pose_python'], '-m', 'egosmplx_pipeline.sapiens_pose', '--repo-root', p['sapiens_repo'],

```

```

        '--image', row['image'], '--output-dir', directory / 'sapiens',
        '--checkpoint', p['pose_checkpoint'], '--checkpoint-sha256', config['checkpoint_sha256']['pose']]
    if p.get('sapiens_dependencies'):
        args += ['--dependencies', p['sapiens_dependencies']]
5     command(args, directory / 'sapiens.log', PACKAGE_ROOT, env)
    row['predictions'] = dict(egosmplx=str(candidates[0]), sapiens=str(directory / 'sapiens/keypoints.json')
)
    command([p['wilor_python'], '-m', 'egosmplx_pipeline.wilor_predict', '--manifest', manifest,
        '--output-dir', observations / 'wilor', '--checkpoint', p['wilor_checkpoint'],
10     '--config', p['wilor_config'], '--detector', p['wilor_detector']],
        observations / 'wilor.log', p['wilor_repo'], env)
    args = [p['pose_python'], '-m', 'egosmplx_pipeline.sapiens_seg', '--repo-root', p['sapiens_repo'],
        '--manifest', manifest, '--output-dir', observations / 'segmentation',
        '--checkpoint', p['seg_checkpoint'], '--checkpoint-sha256', config['checkpoint_sha256']['seg']]
15     if p.get('sapiens_dependencies'):
        args += ['--dependencies', p['sapiens_dependencies']]
    command(args, observations / 'segmentation.log', PACKAGE_ROOT, env)
    for row in observed_rows:
        row['predictions'].update(wilor=str(observations / 'wilor/npz' / (row['id'] + '_wilor.npz')),
20         segmentation=str(observations / 'segmentation/frames' / row['id']))
    write(output / 'automatic_manifest.json', dict(frames=observed_rows))
    return observed_rows
def fingerprint(config, rows):
    import json
25     hashes = {}
    for row in rows:
        for key in ['image', 'calibration']:
            hashes[row['id'] + ':' + key] = sha(row[key])
        for key, value in row.get('predictions', {}).items():
30         if key == 'segmentation':
            for name in ['labels.npy', 'probabilities.npy']:
                hashes[row['id'] + ':' + name] = sha(Path(value) / name)
            else:
                hashes[row['id'] + ':' + key] = sha(value)
35     code = {str(p.relative_to(PACKAGE_ROOT)): sha(p) for p in (PACKAGE_ROOT / 'egosmplx_pipeline'
).rglob('*.py'))}
    return hashlib.sha256(json.dumps(dict(config=config, inputs=hashes, code=code), sort_keys=True).encode()).hexdigest()
def run(config_path, cached=False, resume=False):
40     config, rows = load_config(config_path)
    check = preflight(config, rows, cached)
    output = Path(config['output'])
    if output.exists() and not resume:
        raise FileExistsError('Output exists; choose a new directory or --resume')
45     output.mkdir(parents=True, exist_ok=True)
    with (output / '.lock').open('w') as lock:
        fcntl.flock(lock, fcntl.LOCK_EX | fcntl.LOCK_NB)
        env = environment(config)
        write(output / 'preflight.json', check)
50     signature = fingerprint(config, rows)

```

```

state_path = output / 'run_state.json'
if state_path.exists() and read(state_path)['fingerprint'] != signature:
    raise ValueError('Inputs, code, or configuration changed; use a new output directory')
write(state_path, dict(fingerprint=signature, state='running', cached_predictions=cached))
5 if not cached:
    ready = output / 'automatic_manifest.json'
    if resume and ready.exists():
        rows = read(ready)['frames']
    else:
10         if (output / 'observations').exists():
            stamp = datetime.datetime.now().strftime('%Y%m%dT%H%M%S%f')
            (output / 'observations').rename(output / ('incomplete_observations_' + stamp))
            rows = observe(config, rows, output, env)
records = []
15 for index, row in enumerate(rows):
    destination = output / 'frames' / row['id']
    validation = destination / 'validation.json'
    if resume and validation.exists():
        report = read(validation)
20     valid = report.get('passed') and all((destination / f).is_file() and sha(destination / f) == h
        for f, h in report['artifact_sha256'].items())
    if valid:
        records.append(report)
        print('VERIFIED_RESUME', row['id'], flush=True)
25     continue
    if destination.exists():
        stamp = datetime.datetime.now().strftime('%Y%m%dT%H%M%S%f')
        destination.rename(destination.with_name(destination.name + '_incomplete_' + stamp))
    job_path = output / 'jobs' / (row['id'] + '.json')
30     write(job_path, dict(config=config, row=row, output=str(destination)))
    command([config['paths']['body_python'], '-m', 'egosmplx_pipeline.frame', '--job', job_path,
            output / (row['id'] + '.log'), PACKAGE_ROOT, env)
    records.append(read(validation))
    write(output / 'progress.json', dict(completed=index+1, total=len(rows), last_frame=row['id']))
35 if not all(r.get('pipeline_profile') == selected(config) for r in records):
    job_path = output / 'jobs/reference.json'
    write(job_path, dict(config=config, rows=rows, output=str(output)))
    command([config['paths']['body_python'], '-m', 'egosmplx_pipeline.reference.pipeline', '--job', job_p
ath],
40         output / 'reference.log', PACKAGE_ROOT, env)
    records = [read(output / 'frames' / row['id'] / 'validation.json') for row in rows]
    from egosmplx_pipeline.report import build
    build(output, records)
    write(state_path, dict(fingerprint=signature, state='complete', frame_count=len(records), cached_predi
45 ctions=cached))
    return output
文件名: egosmplx_pipeline/frame.py
功能: 执行两阶段人体拟合, 并准备后续优化的基线融合结果。
"""Execute the two-stage body fit and prepare baseline fusion for refinement."""
50 import argparse

```

```

import json
from pathlib import Path
import sys
import cv2
5 import numpy as np
from egosmplx_pipeline.io import read, write, load, sha, rmse
def canonicalize(parameters, model, camera):
    from egosmplx_pipeline.fusion import geometry
    vertices, joints = geometry(parameters)
10 pixels = camera.world2camera(joints.astype(np.float64)).astype(np.float32)
    parameters.update(smplx_mesh_cam=vertices, smplx_joint_cam=joints, smplx_joint_img=pixels,
                      smplx_joint_cam_root_relative=joints-joints[:1],
                      smplx_body_joints_3d_local=(joints-parameters['transl'].reshape(3))[25],
                      smplx_body_joints_3d_render_cam=joints[25],
15                      smplx_body_joints_2d_render_cam=pixels[25])
    return parameters
def invoke(function, arguments):
    previous = sys.argv
    try:
20     sys.argv = [function.__module__] + list(map(str, arguments))
        function()
    finally:
        sys.argv = previous
def run(config, row, output):
25     from egosmplx_pipeline.bootstrap import setup
    model, Camera = setup(config['paths']['egosmplx_repo'])
    from egosmplx_pipeline import fusion, body_rigid, body_pose
    from egosmplx_pipeline.matching import assign
    from egosmplx_pipeline.render import render, write_obj, read_obj
30     fusion.initialize(model)
    camera = Camera(row['calibration'])
    image = cv2.imread(row['image'])
    if image is None or tuple(camera.img_size) != tuple(image.shape[1::-1]):
        raise ValueError('Image or calibration dimensions invalid')
35     output.mkdir(parents=True, exist_ok=False)
    predictions = row['predictions']
    targets = read(predictions['sapiens'])
    image_digest = sha(row['image'])
    if targets['image_sha256'] != image_digest:
40         raise ValueError('Automatic target image hash differs from input')
    raw = load(predictions['egosmplx'])
    for key in ['global_orient', 'transl']:
        if key + '_network' not in raw:
            raise ValueError('Raw prediction requires explicit original network field: ' + key)
45     raw[key] = raw[key + '_network'].copy()
    # Retain reconstruction inputs and original network rigid outputs only.
    keep = fusion.PARAMS + ['transl', 'global_orient_network', 'transl_network']
    raw = canonicalize({k: raw[k] for k in keep}, model, camera)
    raw_dir = output / 'raw'
50     raw_dir.mkdir()

```

```

np.savez_compressed(raw_dir / 'params.npz', **raw)
write_obj(raw_dir / 'mesh.obj', raw['smplx_mesh_cam'], model.face)
cv2.imwrite(str(raw_dir / 'wireframe.jpg'), render(image, raw['smplx_mesh_cam'].astype(np.float64), model.face, camera, .12))
5   common = ['--repo-root', config['paths']['egosmplx_repo'], '--image', row['image'], '--calibration', row['calibration']]
    invoke(body_rigid.main, common + ['--source-npz', raw_dir / 'params.npz', '--keypoints', predictions['sapiens'],
                                     '--output-dir', output / 'rigid', '--steps', config['fitting']['rigid_steps'], '--device', 'cpu'])
10  invoke(body_pose.main, common + ['--source-npz', output / 'rigid/optimized_params.npz',
                                     '--automatic-keypoints', predictions['sapiens'],
                                     '--output-dir', output / 'body_fit', '--steps', config['fitting']['pose_steps']])
    body = canonicalize(load(output / 'body_fit/automatic/refined_params.npz'), model, camera)
    body_dir = output / 'body'
15  body_dir.mkdir()
    np.savez_compressed(body_dir / 'params.npz', **body)
    write_obj(body_dir / 'mesh.obj', body['smplx_mesh_cam'], model.face)
    cv2.imwrite(str(body_dir / 'wireframe.jpg'), render(image, body['smplx_mesh_cam'].astype(np.float64), model.face, camera, .12))
20  hands = load(predictions['wilor'])
    if 'image_sha256' in hands and str(hands['image_sha256']) != image_digest:
        raise ValueError("WiLoR image hash differs from input")
    if 'focal_length_px' not in hands:
        if 'cached_wilor_focal_px' not in config:
25      raise ValueError("Legacy WiLoR cache requires an explicit verified focal length")
        hands['focal_length_px'] = np.array(config['cached_wilor_focal_px'])
        hands['image_size_wh'] = np.array(image.shape[1::-1])
    if not np.array_equal(hands['image_size_wh'], image.shape[1::-1]):
        raise ValueError("WiLoR image size mismatch")
30  lookup = {q['name']: q for q in targets['keypoints']}
    matches = assign(hands, lookup, body['smplx_joint_img'], *image.shape[1::-1])
    mesh, hand_reports = fusion.stitch(body, hands, matches, camera)
    fused = output / 'fused'
    fused.mkdir()
35  np.savez_compressed(fused / 'mesh.npz', **mesh)
    np.savez_compressed(fused / 'wilor_predictions.npz', **hands)
    write_obj(fused / 'mesh.obj', mesh['vertices_cam'], mesh['faces'])
    write(fused / 'recipe.json', dict(matches=matches, blend_rings=7, body_parameter_path='../body/params.npz',
40      manual_annotations_used=False, local_surface_optimizer=False,
      standard_smplx_parameter_only_reconstruction=False))
    cv2.imwrite(str(fused / 'wireframe.jpg'), render(image, mesh['vertices_cam'].astype(np.float64), mesh['faces'], camera, .12))
    cv2.imwrite(str(fused / 'skeleton.jpg'), fusion.draw_geometry(image, camera, mesh))
45  # Reload actual saved artifacts, including the fusion inputs and recipe.
    reloaded_body = load(body_dir / 'params.npz')
    v, j = fusion.geometry(reloaded_body)
    body_error = float(np.max(np.abs(v-reloaded_body['smplx_mesh_cam'])))
    rebuilt, _ = fusion.stitch(reloaded_body, load(fused / 'wilor_predictions.npz'), read(fused / 'recipe.json')['
50  matches'], camera)

```

```

stored = load(fused / 'mesh.npz')
fusion_error = float(np.max(np.abs(rebuilt['vertices_cam']-stored['vertices_cam'])))
obj_vertices, obj_faces = read_obj(fused / 'mesh.obj')
obj_error = float(np.max(np.abs(obj_vertices-stored['vertices_cam'])))
5 if max(body_error, fusion_error, obj_error) > 1e-5 or not np.array_equal(obj_faces, stored['faces']):
    raise RuntimeError('Saved geometry failed reload validation')
indices = body['stage2_selected_joint_indices'].astype(int)
truth = body['stage2_target_joint_pixels']
body_errors = np.linalg.norm(body['smplx_joint_img'][indices]-truth, axis=1)
10 hand_errors = []
for side, match in matches.items():
    k = fusion.SIDES.index(side)
    projected = camera.world2camera(mesh['hand_joints_cam'][k].astype(np.float64))
    hand_errors.extend(np.linalg.norm(projected-hands['joints_2d'][match['candidate']], axis=1).tolist())
15 report = dict(id=row['id'], passed=True, matched_hands=len(matches), matches=matches,
    body_target_count=len(indices), body_error_px=body_errors.tolist(), hand_error_px=hand_errors,
rs,
    body_rmse_px=rmse(body['smplx_joint_img'][indices], truth),
    hand_rmse_px=float(np.sqrt(np.mean(np.square(hand_errors)))) if hand_errors else None,
20 body_reload_max_abs_m=body_error, fusion_reload_max_abs_m=fusion_error,
    obj_reload_max_abs_m=obj_error, minimum_fused_vertex_z_m=float(mesh['vertices_cam'][:,
2].min()),
    hand_reports=hand_reports, manual_annotations_used=False,
    accuracy_reference='automatic predictions, not human ground truth',
25 input_image_sha256=image_digest, calibration_sha256=sha(row['calibration']))
report['artifact_sha256'] = {str(p.relative_to(output)): sha(p) for p in output.rglob('*') if p.is_file()}
write(output / 'validation.json', report)
print('FRAME_COMPLETE', row['id'], report['body_rmse_px'], report['hand_rmse_px'], flush=True)
return report
30 def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--job', type=Path, required=True)
    args = parser.parse_args()
    job = read(args.job)
35 run(job['config'], job['row'], Path(job['output']))
if __name__ == '__main__':
    main()

```

文件名: egosmplx_pipeline/bootstrap.py

功能: 在工作进程中初始化独立安装的人体重建后端。

```

40 """Initialize the separately installed EgoSMPLX backend in a worker process."""
import os
import sys
from pathlib import Path
def setup(repo):
45     repo = Path(repo).resolve()
    for sub in ['', 'main', 'common', 'data']:
        sys.path.insert(0, str(repo / sub))
    os.environ['EGOSMPLX_REPO_ROOT'] = str(repo)
    from config import cfg
50 path = os.environ.get('EGOSMPLX_CONFIG', str(repo / 'main/config/config_ft_egopw_scenego_ego

```

```

wholebody_unrealego_egofishbody.py'))
    cfg.get_config_fromfile(path)
    from egosmplx_pipeline import camera as calibrated_camera
    sys.modules['FishEyeCalibrated'] = calibrated_camera
5    from egosmplx_pipeline.camera import FishEyeCameraCalibrated
    from egosmplx_pipeline.projection import stable_world2camera
    FishEyeCameraCalibrated.world2camera_pytorch = stable_world2camera
    from utils.human_models import smpl_x
    import torch
10    torch.set_num_threads(2)
    torch.backends.cuda.matmul.allow_tf32 = False
    torch.backends.cudnn.allow_tf32 = False
    return smpl_x, FishEyeCameraCalibrated
文件名: egosmplx_pipeline/io.py
15 功能: 严格读写配置与数组文件, 记录结果复现所需元数据。
    """Strict JSON/NPZ I/O and reproducibility metadata."""
    import hashlib
    import json
    from pathlib import Path
20    import numpy as np
    def read(path):
        return json.loads(Path(path).read_text(encoding='utf-8'))
    def write(path, value):
        path = Path(path)
25    path.parent.mkdir(parents=True, exist_ok=True)
        temporary = path.with_suffix(path.suffix + '.tmp')
        temporary.write_text(json.dumps(value, indent=2, ensure_ascii=False, allow_nan=False) + '\n', encoding
='utf-8')
        temporary.replace(path)
30    def sha(path):
        h = hashlib.sha256()
        with Path(path).open('rb') as handle:
            for block in iter(lambda: handle.read(8 * 1024**2), b''):
                h.update(block)
35    return h.hexdigest()
    def load(path):
        with np.load(path, allow_pickle=False) as data:
            result = {k: data[k].copy() for k in data.files}
            for key, value in result.items():
40                if np.issubdtype(value.dtype, np.number) and not np.isfinite(value).all():
                    raise ValueError('Nonfinite NPZ field: ' + key)
            return result
    def arr(tensor):
        return tensor.detach().cpu().numpy()
45    def rmse(prediction, target):
        """Root mean squared 2D Euclidean distance, not coordinate-wise RMSE."""
        prediction, target = np.asarray(prediction), np.asarray(target)
        if prediction.shape != target.shape or prediction.ndim != 2 or prediction.shape[1] != 2:
            raise ValueError('Expected matching N x 2 arrays')
50    if not len(prediction):

```

```

        return None
    return float(np.sqrt(np.mean(np.sum((prediction - target)**2, axis=1))))
文件名: egosmplx_pipeline/sampling.py
功能: 从视频中均匀抽取图像帧, 不将抽帧视为双目同步证明。
5  """Decode evenly spaced video frames; do not claim stereo synchronization."""
    from pathlib import Path
    import re
    import cv2
    from egosmplx_pipeline.io import write, sha
10  def sample_video(video, calibration, camera, count, output):
        if count < 1 or not re.fullmatch('[A-Za-z0-9_-]+', camera):
            raise ValueError('Positive frame count and safe camera name required')
        output = Path(output)
        if output.exists():
15             raise FileExistsError(output)
        cap = cv2.VideoCapture(str(video))
        try:
            n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
            fps = cap.get(cv2.CAP_PROP_FPS)
20             if not cap.isOpened() or n < count or fps <= 0:
                raise ValueError('Video cannot provide requested frames')
            output.mkdir(parents=True)
            rows = []
            for k in range(count):
25                 index = min(n-1, int((k+.5) * n/count))
                cap.set(cv2.CAP_PROP_POS_FRAMES, index)
                ok, image = cap.read()
                if not ok:
                    raise RuntimeError('Failed decoding frame ' + str(index))
30                 identifier = camera + '_f' + str(index).zfill(8)
                target = output / (identifier + '.png')
                if not cv2.imwrite(str(target), image):
                    raise IOError(target)
                rows.append(dict(id=identifier, camera=camera, image=target.name,
35                             calibration=str(Path(calibration).resolve()), image_sha256=sha(target),
                             frame_index_zero_based=index, approximate_video_seconds=index/fps))
            write(output / 'manifest.json', dict(frames=rows, exact_stereo_synchronization_claimed=False))
            return output / 'manifest.json'
        finally:
40             cap.release()
文件名: egosmplx_pipeline/egosmplx_predict.py
功能: 在下视双目抽帧图像上执行人体检测与推理,
    并完成鱼眼相机投影。
#!/usr/bin/env python
45  """Run person detection, EgoSMPLX inference, and fisheye projection on DownwardStereo extracted frames."""
    import argparse
    import json
    import os
50    from pathlib import Path

```

```

import sys
import time
import cv2
import numpy as np
5 import torch
import torchvision.transforms as transforms
from tqdm import tqdm
from mmdet.apis import init_detector, inference_detector
REPO_ROOT = Path(os.environ["EGOSMPLX_REPO_ROOT"]).resolve()
10 from egosmplx_pipeline import camera as calibrated_camera
sys.modules["FishEyeCalibrated"] = calibrated_camera
sys.path.insert(0, str(REPO_ROOT))
sys.path.insert(0, str(REPO_ROOT / "main"))
sys.path.insert(0, str(REPO_ROOT / "data"))
15 sys.path.insert(0, str(REPO_ROOT / "common"))
from config import cfg
from utils.inference_utils import process_mmdet_results, non_max_suppression
def parse_args():
    parser = argparse.ArgumentParser(description=__doc__)
20     parser.add_argument("--image-dir", type=Path, default=REPO_ROOT / "DownwardStereo" / "left_frames_stride10")
    parser.add_argument("--image-glob", default="*.jpg")
    parser.add_argument("--fisheye-calibration", type=Path, default=REPO_ROOT / "DownwardStereo" / "fisheye.left.calibration.json")
25     parser.add_argument(
        "--config-path",
        type=Path,
        default=REPO_ROOT / "main" / "config" / "config_ft_egopw_sceneego_egowholebody_unrealego_egofishbody.py",
30     )
    parser.add_argument(
        "--checkpoint-path",
        type=Path,
        default=REPO_ROOT / "output" / "train_5datasets_4gpu_resume_s7_to_s12_20260801_191616" / "model_dump" / "snapshot_11.pth.tar",
35     )
    parser.add_argument(
        "--det-config",
        type=Path,
        default=REPO_ROOT / "pretrained_models" / "mmdet" / "mmdet_faster_rcnn_r50_fpn_coco.py",
40     )
    parser.add_argument(
        "--det-checkpoint",
        type=Path,
        default=REPO_ROOT / "pretrained_models" / "mmdet" / "faster_rcnn_r50_fpn_1x_coco_20200130-047c8118.pth",
45     )
    parser.add_argument("--output-dir", type=Path, default=None)
    parser.add_argument("--max-images", type=int, default=None)
50     parser.add_argument("--score-thr", type=float, default=0.3)

```

```

    parser.add_argument("--bbox-thr", type=float, default=50.0)
    parser.add_argument("--iou-thr", type=float, default=0.5)
    parser.add_argument("--multi-person", action="store_true")
    parser.add_argument("--no-detector", action="store_true", help="Run one full-image SMPL-X predictio
5   n per image without Faster R-CNN.")
    parser.add_argument("--save-mesh", action="store_true")
    parser.add_argument(
        "--save-wireframe-overlay",
        action="store_true",
10    help="Also save a wireframe overlay from the same prediction and camera projection.",
    )
    parser.add_argument("--save-video", action="store_true")
    parser.add_argument("--show-verts", action="store_true")
    parser.add_argument("--show-bbox", action="store_true", default=True)
15    parser.add_argument("--mesh-mode", choices=("solid", "wireframe", "points"), default="solid")
    parser.add_argument(
        "--smplx-input-mode",
        choices=(
            "full_image",
20    "det_crop",
            "det_tangent",
            "det_undistort_contiguous",
        ),
        default="full_image",
25    help=(
        "det_tangent uses the detected person box to place the 16x12 ViT "
        "tangent patches in original fisheye-camera coordinates; "
        "det_undistort_contiguous first undistorts the full frame, detects "
        "there, and samples one continuous 12-column x 16-row ViT crop."
30    ),
    )
    parser.add_argument(
        "--det-tangent-expand",
        type=float,
35    default=1.2,
        help="Isotropic expansion applied to the detector box in det_tangent mode.",
    )
    parser.add_argument(
        "--det-tangent-patch-size",
40    type=float,
        default=0.1,
        help="Half-size of each tangent-plane patch in det_tangent mode.",
    )
    parser.add_argument(
        "--global-undistort-alpha",
45    type=float,
        default=1.0,
        help=(
        "OpenCV optimal-new-camera alpha for det_undistort_contiguous: "
50    "0 crops invalid borders, 1 retains the full calibrated field."

```

```

    ),
)
parser.add_argument(
    "--contiguous-roi-mode",
5    choices=("stretch", "aspect_pad"),
    default="stretch",
    help=(
        "For det_undistort_contiguous, stretch the clipped expanded box "
        "to the 12:16 token canvas, or preserve 12:16 geometry with "
10    "out-of-image padding."
    ),
)
parser.add_argument(
    "--input-crop-mode",
15    choices=("none", "center_square", "training_horizontal"),
    default="none",
    help="Crop the network input without changing the original image used for fisheye mesh overlay.",
)
parser.add_argument(
20    "--horizontal-crop-pixels",
    type=int,
    default=128,
    help="Pixels removed from both sides with --input-crop-mode training_horizontal.",
)
25 parser.add_argument("--contact-sheet-limit", type=int, default=40)
parser.add_argument(
    "--refine-camera-from-positionnet",
    action="store_true",
    help=(
30    "Keep the predicted SMPL-X local pose/shape fixed and refine a "
        "camera extrinsic rotation/translation against PositionNet upper-body joints."
    ),
)
parser.add_argument("--camera-fit-iters", type=int, default=300)
35 parser.add_argument("--camera-fit-lr", type=float, default=0.03)
parser.add_argument(
    "--camera-fit-mode",
    choices=("translation", "translation_global_orient", "extrinsics"),
    default="translation",
40    help=(
        "With --refine-camera-from-positionnet, optimize tx,ty,tz only; "
        "SMPL-X global_orient plus translation; or the legacy camera-frame "
        "rotation plus translation."
    ),
)
45 )
parser.add_argument("--num-gpus", type=int, default=1)
return parser.parse_args()
def make_output_dir(args):
    if args.output_dir is not None:
50    return args.output_dir

```

```

stamp = time.strftime("%Y%m%d_%H%M%S")
suffix = "full" if args.max_images is None else f"{args.max_images}imgs"
return REPO_ROOT / "output" / f"downwardstereo_left_stride10_detect_s11_{suffix}_{stamp}"

def setup_model(args, output_dir):
5   os.environ.setdefault("EGOSMPLX_REPO_ROOT", str(REPO_ROOT))
   os.environ["EGOSMPLX_FISHEYE_CALIBRATION"] = str(args.fisheye_calibration.resolve())
   cfg.get_config_fromfile(str(args.config_path.resolve()))
   cfg.update_test_config(
       testset="EHF",
10      agora_benchmark="na",
       shapy_eval_split=None,
       pretrained_model_path=str(args.checkpoint_path.resolve()),
       use_cache=False,
   )
15  cfg.fisheye_camera_path = str(args.fisheye_calibration.resolve())
   cfg.fisheye_camera_paths = [str(args.fisheye_calibration.resolve())]
   cfg.update_config(args.num_gpus, str(output_dir))
   from base import Demoer
   demoer = Demoer()
20  demoer._make_model()
   demoer.model.eval()
   return demoer

def load_runtime_utils():
   from FishEyeCalibrated import FishEyeCameraCalibrated
25  from utils.human_models import smpl_x
   from utils.preprocessing import load_img, process_bbox, generate_patch_image
   from utils.vis import render_mesh_fisheye, save_obj
   return FishEyeCameraCalibrated, smpl_x, load_img, process_bbox, generate_patch_image, render_mesh
   _fisheye, save_obj

30  def filter_bboxes(raw_bboxes, score_thr, bbox_thr):
   kept = []
   for bbox in raw_bboxes:
       x1, y1, x2, y2, score = [float(v) for v in bbox[:5]]
       w = abs(x2 - x1)
35      h = abs(y2 - y1)
       if score < score_thr:
           continue
       if w < bbox_thr or h < bbox_thr:
           continue
40      kept.append(np.array([x1, y1, x2, y2, score], dtype=np.float32))
   return kept

def save_contact_sheet(image_paths, out_path, max_images=40, thumb_w=360):
   paths = image_paths[:max_images]
   if not paths:
45      return
   thumbs = []
   for path in paths:
       img = cv2.imread(str(path), cv2.IMREAD_COLOR)
       if img is None:
50      continue

```

```

        h, w = img.shape[:2]
        thumb_h = max(1, int(h * thumb_w / max(1, w)))
        thumbs.append(cv2.resize(img, (thumb_w, thumb_h), interpolation=cv2.INTER_AREA))
    if not thumbs:
5         return
    cols = min(4, len(thumbs))
    rows = int(np.ceil(len(thumbs) / cols))
    max_h = max(t.shape[0] for t in thumbs)
    canvas = np.full((rows * max_h, cols * thumb_w, 3), 255, dtype=np.uint8)
10    for i, thumb in enumerate(thumbs):
        r, c = divmod(i, cols)
        canvas[r * max_h : r * max_h + thumb.shape[0], c * thumb_w : c * thumb_w + thumb.shape[1]] = thumb
    mb
    cv2.imwrite(str(out_path), canvas)
15 def draw_no_detection(img_rgb, text="NO PERSON DETECTED"):
    img = img_rgb.copy()
    cv2.putText(img, text, (40, 70), cv2.FONT_HERSHEY_SIMPLEX, 1.4, (255, 40, 40), 3, cv2.LINE_AA
)
    return img
20 def rgb_to_bgr_u8(img_rgb):
    img = np.clip(img_rgb, 0, 255).astype(np.uint8)
    return img[:, :, ::-1]
def expand_bbox_xyxy(box_xyxy, scale, image_w, image_h):
    """Expand a detector box about its center and clip it to the source image."""
25    x1, y1, x2, y2 = [float(value) for value in box_xyxy[:4]]
    center_x = (x1 + x2) * 0.5
    center_y = (y1 + y2) * 0.5
    width = max(2.0, abs(x2 - x1) * float(scale))
    height = max(2.0, abs(y2 - y1) * float(scale))
30    x1 = np.clip(center_x - width * 0.5, 0.0, float(image_w - 1))
    x2 = np.clip(center_x + width * 0.5, 0.0, float(image_w - 1))
    y1 = np.clip(center_y - height * 0.5, 0.0, float(image_h - 1))
    y2 = np.clip(center_y + height * 0.5, 0.0, float(image_h - 1))
    if x2 - x1 < 2.0 or y2 - y1 < 2.0:
35        raise ValueError(f"Expanded detector box is degenerate: {(x1, y1, x2, y2)}")
    return np.asarray([x1, y1, x2, y2], dtype=np.float32)
def create_global_undistort_model(calibration_path, image_w, image_h, alpha):
    """Build one continuous Brown-Conrady rectification for the full frame."""
    calibration = json.loads(Path(calibration_path).read_text())
40    required = (
        "sourceFocalLength",
        "sourcePrincipalPoint",
        "sourceRadialDistortion",
    )
45    missing = [key for key in required if key not in calibration]
    if missing:
        raise KeyError(
            "Continuous global undistortion requires Brown-Conrady source "
            f"fields in {calibration_path}; missing {missing}"
50    )

```

```

calibration_size = tuple(int(value) for value in calibration.get("size", (image_w, image_h)))
if calibration_size != (int(image_w), int(image_h)):
    raise ValueError(
        "Calibration/image size mismatch for global undistortion: "
5         f'calibration={calibration_size}, image={{image_w, image_h}}'
    )
focal = np.asarray(calibration["sourceFocalLength"], dtype=np.float64)
principal = np.asarray(calibration["sourcePrincipalPoint"], dtype=np.float64)
radial = list(calibration["sourceRadialDistortion"])
10 tangential = list(calibration.get("sourceTangentialDistortion", (0.0, 0.0)))
while len(radial) < 3:
    radial.append(0.0)
source_k = np.asarray(
    [
15     [focal[0], 0.0, principal[0]],
        [0.0, focal[1], principal[1]],
        [0.0, 0.0, 1.0],
    ],
    dtype=np.float64,
20 )
distortion = np.asarray(
    [radial[0], radial[1], tangential[0], tangential[1], radial[2]],
    dtype=np.float64,
)
25 new_k, valid_roi = cv2.getOptimalNewCameraMatrix(
    source_k,
    distortion,
    (int(image_w), int(image_h)),
    float(alpha),
30 (int(image_w), int(image_h)),
    centerPrincipalPoint=False,
)
map_x, map_y = cv2.initUndistortRectifyMap(
    source_k,
35 distortion,
    None,
    new_k,
    (int(image_w), int(image_h)),
    cv2.CV_32FC1,
40 )
return {
    "source_k": source_k,
    "distortion": distortion,
    "new_k": new_k,
45 "valid_roi": tuple(int(value) for value in valid_roi),
    "map_x": map_x,
    "map_y": map_y,
    "alpha": float(alpha),
}
50 def undistort_full_image(image_rgb, undistort_model):

```

```

        """Remap a complete fisheye frame into one continuous pinhole image."""
        return cv2.remap(
            image_rgb,
            undistort_model["map_x"],
5            undistort_model["map_y"],
            interpolation=cv2.INTER_LINEAR,
            borderMode=cv2.BORDER_CONSTANT,
            borderValue=(0, 0, 0),
        )
10    def undistorted_pixels_to_original(points_xy, undistort_model):
        """Map pixels from the continuous undistorted view to the raw frame."""
        points = np.asarray(points_xy, dtype=np.float64).reshape(-1, 2)
        new_k = undistort_model["new_k"]
        normalized = np.column_stack(
15            (
                (points[:, 0] - new_k[0, 2]) / new_k[0, 0],
                (points[:, 1] - new_k[1, 2]) / new_k[1, 1],
                np.ones((len(points),), dtype=np.float64),
            )
20        )
        projected, _ = cv2.projectPoints(
            normalized,
            np.zeros((3, 1), dtype=np.float64),
            np.zeros((3, 1), dtype=np.float64),
25            undistort_model["source_k"],
            undistort_model["distortion"],
        )
        return projected.reshape(-1, 2).astype(np.float32)
    def expand_bbox_to_aspect(box_xyxy, scale, target_width_over_height):
30        """Expand a box and pad it to one aspect ratio without clipping."""
        x1, y1, x2, y2 = [float(value) for value in box_xyxy[:4]]
        center_x = (x1 + x2) * 0.5
        center_y = (y1 + y2) * 0.5
        width = max(2.0, abs(x2 - x1) * float(scale))
35        height = max(2.0, abs(y2 - y1) * float(scale))
        if width / height > float(target_width_over_height):
            height = width / float(target_width_over_height)
        else:
            width = height * float(target_width_over_height)
40        return np.asarray(
            [
                center_x - width * 0.5,
                center_y - height * 0.5,
                center_x + width * 0.5,
45                center_y + height * 0.5,
            ],
            dtype=np.float32,
        )
    def create_contiguous_bbox_pixel_grid(
50        bbox_xyxy,

```

```

    patch_rows,
    patch_columns,
    patch_h,
    patch_w,
5   ):
    """Split one continuous rectified ROI into row-major ViT patch grids."""
    x1, y1, x2, y2 = [float(value) for value in bbox_xyxy]
    output_h = int(patch_rows) * int(patch_h)
    output_w = int(patch_columns) * int(patch_w)
10   xs = np.linspace(x1, x2, output_w, dtype=np.float32)
    ys = np.linspace(y1, y2, output_h, dtype=np.float32)
    grid_x, grid_y = np.meshgrid(xs, ys)
    full_grid = np.stack((grid_x, grid_y), axis=-1)
    return (
15       full_grid.reshape(
           patch_rows,
           patch_h,
           patch_columns,
           patch_w,
20       2,
       )
       .transpose(0, 2, 1, 3, 4)
       .reshape(patch_rows * patch_columns, patch_h, patch_w, 2)
       .astype(np.float32)
25   )
def draw_contiguous_sampling_grid(image_rgb, grid_pixels, bbox_xyxy):
    """Draw the regular 12-column x 16-row crop on the undistorted image."""
    canvas = image_rgb.copy()
    for patch in grid_pixels:
30       corners = np.asarray(
           [patch[0, 0], patch[0, -1], patch[-1, -1], patch[-1, 0]],
           dtype=np.float32,
       )
       cv2.polylines(
35           canvas,
           [np rint(corners).astype(np.int32)],
           isClosed=True,
           color=(255, 0, 0),
           thickness=1,
40           lineType=cv2.LINE_AA,
       )
       x1, y1, x2, y2 = np rint(bbox_xyxy).astype(int)
       cv2.rectangle(canvas, (x1, y1), (x2, y2), (255, 0, 0), 3)
       cv2.putText(
45           canvas,
           "continuous undistorted ROI -> 12 cols x 16 rows",
           (max(10, x1), max(30, y1 - 12)),
           cv2.FONT_HERSHEY_SIMPLEX,
           0.68,
50           (255, 0, 0),

```

```

    2,
    cv2.LINE_AA,
)
return canvas
5 def create_bbox_tangent_pixel_grid(
    sampler,
    fisheye_camera,
    bbox_xyxy,
    patch_rows=16,
10    patch_columns=12,
    patch_size=0.1,
):
    """Create row-major tangent-patch samples in original fisheye pixels."""
    x1, y1, x2, y2 = [float(value) for value in bbox_xyxy]
15    horizontal = (np.arange(patch_columns, dtype=np.float32) + 0.5) / patch_columns
    vertical = (np.arange(patch_rows, dtype=np.float32) + 0.5) / patch_rows
    normalized_centers = np.dstack(np.meshgrid(horizontal, vertical)).reshape(-1, 2)
    centers_2d = normalized_centers.copy()
    centers_2d[:, 0] = x1 + centers_2d[:, 0] * (x2 - x1)
20    centers_2d[:, 1] = y1 + centers_2d[:, 1] * (y2 - y1)
    centers_2d_up = centers_2d.copy()
    centers_2d_up[:, 1] -= (y2 - y1) / (2.0 * patch_rows)
    depth = np.ones((len(centers_2d),), dtype=np.float32)
    centers_3d = fisheye_camera.camera2world(centers_2d, depth)
25    centers_3d_up = fisheye_camera.camera2world(centers_2d_up, depth)
    patch_h, patch_w = [
        int(value) for value in sampler.patches_2d_per_camera_pixels.shape[-3:-1]
    ]
    tangent_patches = [
30        sampler._generate_patch_coordinates(
            center,
            center_up,
            (float(patch_size), float(patch_size)),
            (patch_h, patch_w),
35            visualize=False,
        )
        for center, center_up in zip(centers_3d, centers_3d_up)
    ]
    grid = sampler._project_patches_to_original_image(
40        tangent_patches,
        fisheye_camera=fisheye_camera,
        image_h=int(fisheye_camera.img_size[1]),
        image_w=int(fisheye_camera.img_size[0]),
        normalize=False,
45    )
    return np.asarray(grid, dtype=np.float32)
def draw_tangent_sampling_grid(image_rgb, grid_pixels, bbox_xyxy):
    """Draw every actual tangent-patch footprint and its detected ROI."""
    canvas = image_rgb.copy()
50    for patch in grid_pixels:

```

```

    corners = np.asarray(
        [patch[0, 0], patch[0, -1], patch[-1, -1], patch[-1, 0]],
        dtype=np.float32,
    )
5   if np.isfinite(corners).all():
        cv2.polylines(
            canvas,
            [np rint(corners).astype(np.int32)],
            True,
10         (0, 255, 0),
            1,
            cv2.LINE_AA,
        )
    x1, y1, x2, y2 = np rint(bbox_xyxy).astype(int)
15   cv2.rectangle(canvas, (x1, y1), (x2, y2), (255, 0, 0), 3)
    cv2.putText(
        canvas,
        "det ROI -> 16x12 tangent patches",
        (max(10, x1), max(30, y1 - 12)),
20     cv2.FONT_HERSHEY_SIMPLEX,
        0.75,
        (255, 0, 0),
        2,
        cv2.LINE_AA,
25   )
    return canvas
def save_vit_patch_atlas(
    sampled_patches,
    out_path,
30   rows=16,
    columns=12,
    scale=4,
    gap=2,
    grid_color_rgb=None,
35   ):
    """Save the exact RGB patches returned by UndistortPatch before PatchEmbed."""
    patches = sampled_patches.detach().cpu().numpy()[0]
    if patches.shape[0] != rows * columns:
        raise ValueError(
40         f'Expected {rows * columns} ViT patches, got {patches.shape[0]}'
        )
    patches = np.clip(patches.transpose(0, 2, 3, 1) * 255.0, 0, 255).astype(np.uint8)
    patch_h, patch_w = patches.shape[1:3]
    atlas_h = rows * patch_h + (rows - 1) * gap
45   atlas_w = columns * patch_w + (columns - 1) * gap
    atlas = np.zeros((atlas_h, atlas_w, 3), dtype=np.uint8)
    for patch_index, patch in enumerate(patches):
        row, column = divmod(patch_index, columns)
        top = row * (patch_h + gap)
50     left = column * (patch_w + gap)

```

```

    atlas[top:top + patch_h, left:left + patch_w] = patch
atlas = cv2.resize(
    atlas,
    (atlas.shape[1] * scale, atlas.shape[0] * scale),
5    interpolation=cv2.INTER_NEAREST,
)
if grid_color_rgb is not None:
    patch_h_scaled = patch_h * scale
    patch_w_scaled = patch_w * scale
10    for row in range(1, rows):
        y = row * patch_h_scaled
        cv2.line(
            atlas,
            (0, y),
15            (atlas.shape[1] - 1, y),
            grid_color_rgb,
            2,
            cv2.LINE_AA,
        )
20    for column in range(1, columns):
        x = column * patch_w_scaled
        cv2.line(
            atlas,
            (x, 0),
25            (x, atlas.shape[0] - 1),
            grid_color_rgb,
            2,
            cv2.LINE_AA,
        )
30    cv2.imwrite(str(out_path), rgb_to_bgr_u8(atlas))
POSITIONNET_BODY_EDGES = (
    (0, 1), (1, 3), (3, 5),
    (0, 2), (2, 4), (4, 6),
    (0, 7),
35    (7, 8), (8, 10), (10, 12),
    (7, 9), (9, 11), (11, 13),
    (5, 14), (5, 15), (5, 16),
    (6, 17), (6, 18), (6, 19),
    (7, 24), (24, 22), (22, 20), (24, 23), (23, 21),
40 )
def positionnet_coords_to_sampling_grid(
    body_joint_img,
    sampling_grid_pixels,
    patch_rows=16,
45    patch_columns=12,
):
    """Map PositionNet coordinates through the actual tangent-patch centers."""
    coords = np.asarray(body_joint_img, dtype=np.float32)
    grid = np.asarray(sampling_grid_pixels, dtype=np.float32)
50    if grid.shape[0] != patch_rows * patch_columns:

```

```

        raise ValueError(
            f'Expected {patch_rows * patch_columns} sampling patches, got {grid.shape[0]}'
        )
    patch_h, patch_w = grid.shape[1:3]
5   center_rows = slice((patch_h - 1) // 2, patch_h // 2 + 1)
    center_columns = slice((patch_w - 1) // 2, patch_w // 2 + 1)
    patch_centers = grid.reshape(
        patch_rows, patch_columns, patch_h, patch_w, 2
    )[:, :, center_rows, center_columns].mean(axis=(2, 3))
10  x = np.clip(coords[:, 0], 0.0, float(patch_columns - 1))
    y = np.clip(coords[:, 1], 0.0, float(patch_rows - 1))
    x0 = np.floor(x).astype(np.int64)
    y0 = np.floor(y).astype(np.int64)
    x1 = np.minimum(x0 + 1, patch_columns - 1)
15  y1 = np.minimum(y0 + 1, patch_rows - 1)
    wx = (x - x0).reshape(-1, 1)
    wy = (y - y0).reshape(-1, 1)
    pixels = (
        (1.0 - wx) * (1.0 - wy) * patch_centers[y0, x0]
20  + wx * (1.0 - wy) * patch_centers[y0, x1]
        + (1.0 - wx) * wy * patch_centers[y1, x0]
        + wx * wy * patch_centers[y1, x1]
    )
    return pixels.astype(np.float32), patch_centers.astype(np.float32)
25  def draw_positionnet_result(image_rgb, body_joint_hm, body_joint_pixels, sampling_bbox_xyxy):
    """Overlay the depth-marginalized PositionNet heatmap and body skeleton."""
    canvas = image_rgb.copy()
    heatmap_2d = np.asarray(body_joint_hm, dtype=np.float32).sum(axis=(0, 1))
    heatmap_2d -= heatmap_2d.min()
30  peak = float(heatmap_2d.max())
    if peak > 0:
        heatmap_2d /= peak
        heatmap_color_bgr = cv2.applyColorMap(
            np rint(heatmap_2d * 255.0).astype(np.uint8),
35  cv2.COLORMAP_JET,
        )
        heatmap_color = heatmap_color_bgr[:, :, ::-1]
        image_h, image_w = canvas.shape[:2]
        x1, y1, x2, y2 = [int(round(value)) for value in sampling_bbox_xyxy]
40  x1, x2 = np.clip([x1, x2], 0, image_w)
        y1, y2 = np.clip([y1, y2], 0, image_h)
        if x2 > x1 and y2 > y1:
            heatmap_roi = cv2.resize(
                heatmap_color,
45  (x2 - x1, y2 - y1),
                interpolation=cv2.INTER_CUBIC,
            ).astype(canvas.dtype, copy=False)
            canvas[y1:y2, x1:x2] = cv2.addWeighted(
                canvas[y1:y2, x1:x2],
50  0.68,

```

```

        heatmap_roi,
        0.32,
        0.0,
    )
5   for start, end in POSITIONNET_BODY_EDGES:
        points = body_joint_pixels[[start, end]]
        if np.isfinite(points).all():
            cv2.line(
                canvas,
10            tuple(np rint(points[0]).astype(int)),
                tuple(np rint(points[1]).astype(int)),
                (255, 255, 0),
                4,
                cv2.LINE_AA,
15            )
        for point in body_joint_pixels:
            if np.isfinite(point).all():
                cv2.circle(
                    canvas,
20                tuple(np rint(point).astype(int)),
                    6,
                    (255, 0, 0),
                    -1,
                    cv2.LINE_AA,
25                )
            cv2.putText(
                canvas,
                "PositionNet: 25 body joints (yellow) + heatmap",
                (24, 42),
30                cv2.FONT_HERSHEY_SIMPLEX,
                0.8,
                (255, 255, 0),
                2,
                cv2.LINE_AA,
35            )
        return canvas

def draw_positionnet_smplx_2d_comparison(
    image_rgb,
    positionnet_pixels,
40    smplx_pixels,
):
    """Draw all 25 corresponding body joints from PositionNet and SMPL-X."""
    canvas = image_rgb.copy()
    positionnet_pixels = np.asarray(positionnet_pixels, dtype=np.float32)
    smplx_pixels = np.asarray(smplx_pixels, dtype=np.float32)
45    if positionnet_pixels.shape != (25, 2) or smplx_pixels.shape != (25, 2):
        raise ValueError(
            "PositionNet and SMPL-X body points must both have shape (25, 2)"
        )
50    valid = np.isfinite(positionnet_pixels).all(axis=1) & np.isfinite(smplx_pixels).all(axis=1)

```

```

valid &= (np.abs(positionnet_pixels) < 1e6).all(axis=1)
valid &= (np.abs(smplx_pixels) < 1e6).all(axis=1)
errors = np.linalg.norm(smplx_pixels - positionnet_pixels, axis=1)
# Draw per-joint correspondence first so the two skeletons remain visible.
5   for joint_index in np.flatnonzero(valid):
        positionnet_point = tuple(np rint(positionnet_pixels[joint_index]).astype(int))
        smplx_point = tuple(np rint(smplx_pixels[joint_index]).astype(int))
        cv2.line(
            canvas,
10         positionnet_point,
            smplx_point,
            (0, 255, 255),
            1,
            cv2.LINE_AA,
15     )
    for start, end in POSITIONNET_BODY_EDGES:
        if valid[start] and valid[end]:
            cv2.line(
                canvas,
20         tuple(np rint(positionnet_pixels[start]).astype(int)),
                tuple(np rint(positionnet_pixels[end]).astype(int)),
                (255, 0, 0),
                3,
                cv2.LINE_AA,
25     )
            cv2.line(
                canvas,
                tuple(np rint(smplx_pixels[start]).astype(int)),
                tuple(np rint(smplx_pixels[end]).astype(int)),
30         (0, 255, 0),
                3,
                cv2.LINE_AA,
            )
    for joint_index in np.flatnonzero(valid):
35         positionnet_point = tuple(np rint(positionnet_pixels[joint_index]).astype(int))
            smplx_point = tuple(np rint(smplx_pixels[joint_index]).astype(int))
            cv2.circle(canvas, positionnet_point, 6, (255, 0, 0), -1, cv2.LINE_AA)
            cv2.circle(canvas, smplx_point, 7, (0, 255, 0), 2, cv2.LINE_AA)
            midpoint = tuple(
40         np rint((positionnet_pixels[joint_index] + smplx_pixels[joint_index]) * 0.5)
                .astype(int)
            )
            cv2.putText(
                canvas,
45         str(joint_index),
                midpoint,
                cv2.FONT_HERSHEY_SIMPLEX,
                0.42,
                (255, 255, 255),
50         1,

```

```

        cv2.LINE_AA,
    )
    selected_valid = valid[CAMERA_FIT_BODY_INDICES]
    selected_errors = errors[CAMERA_FIT_BODY_INDICES][selected_valid]
5    all_mean_error = float(errors[valid].mean()) if valid.any() else float("nan")
    fit_mean_error = (
        float(selected_errors.mean()) if len(selected_errors) else float("nan")
    )
    labels = (
10    ((255, 0, 0), "red: PositionNet 2D (filled)"),
        ((0, 255, 0), "green: SMPL-X projected 2D (ring)"),
        ((0, 255, 255), "cyan: corresponding joint"),
        ((255, 255, 255), f"mean all25={all_mean_error:.1f}px, fit12={fit_mean_error:.1f}px"),
    )
15    for row, (color, label) in enumerate(labels):
        cv2.putText(
            canvas,
            label,
            (24, 38 + row * 31),
20            cv2.FONT_HERSHEY_SIMPLEX,
            0.68,
            color,
            2,
            cv2.LINE_AA,
25        )
    return canvas

CAMERA_FIT_BODY_INDICES = np.asarray(
    [7, 8, 9, 10, 11, 12, 13, 20, 21, 22, 23, 24], dtype=np.int64
)
30 CAMERA_FIT_BODY_EDGES = (
    (7, 8), (7, 9), (8, 10), (10, 12), (9, 11), (11, 13),
    (7, 20), (7, 21), (20, 22), (21, 23), (22, 24), (23, 24),
)
def heatmap_joints_to_original_pixels(joint_img, input_hw, input_offset):
35    """Invert the normalization used by Model.get_coord for full-image input."""
    input_h, input_w = [float(v) for v in input_hw]
    offset_x, offset_y = [float(v) for v in input_offset]
    joint_px = np.asarray(joint_img, dtype=np.float64).copy()
    joint_px[:, 0] = joint_px[:, 0] / float(cfg.output_hm_shape[2]) * input_w + offset_x
40    joint_px[:, 1] = joint_px[:, 1] / float(cfg.output_hm_shape[1]) * input_h + offset_y
    return joint_px

def axis_angle_to_matrix(axis_angle):
    """Differentiable Rodrigues transform for a single three-vector."""
    theta = torch.linalg.vector_norm(axis_angle).clamp_min(1e-8)
45    axis = axis_angle / theta
    x, y, z = axis.unbind()
    zero = torch.zeros((), device=axis.device, dtype=axis.dtype)
    skew = torch.stack(
        (zero, -z, y, z, zero, -x, -y, x, zero)
50    ).reshape(3, 3)

```

```

identity = torch.eye(3, device=axis.device, dtype=axis.dtype)
return identity + torch.sin(theta) * skew + (1.0 - torch.cos(theta)) * (skew @ skew)
def compose_global_orient(delta_rotation, network_global_orient):
    """Left-compose a camera-frame delta into the SMPL-X root axis-angle."""
5    delta_np = delta_rotation.detach().cpu().numpy().astype(np.float64)
    network_np = (
        network_global_orient.detach().cpu().numpy().reshape(3).astype(np.float64)
    )
    network_rotation, _ = cv2.Rodrigues(network_np)
10    refined_rotation = delta_np @ network_rotation
    refined_axis_angle, _ = cv2.Rodrigues(refined_rotation)
    return torch.as_tensor(
        refined_axis_angle.reshape(3),
        device=network_global_orient.device,
15    dtype=network_global_orient.dtype,
    )
def refine_camera_extrinsics(
    body_joints_cam,
    body_targets_px,
20    initial_translation,
    fisheye_cam,
    iterations,
    learning_rate,
):
25    """Fit a camera-frame delta R,t while local pose, shape, and expression stay fixed."""
    if iterations <= 0:
        raise ValueError("--camera-fit-iters must be positive")
    if learning_rate <= 0:
        raise ValueError("--camera-fit-lr must be positive")
30    device = body_joints_cam.device
    dtype = body_joints_cam.dtype
    indices = torch.as_tensor(CAMERA_FIT_BODY_INDICES, device=device, dtype=torch.long)
    joints = body_joints_cam.index_select(0, indices).detach()
    targets = torch.as_tensor(
35    body_targets_px[CAMERA_FIT_BODY_INDICES], device=device, dtype=dtype
    )
    initial = initial_translation.detach().to(device=device, dtype=dtype)
    identity = torch.eye(3, device=device, dtype=dtype)
    def project(rotation, translation):
40    transformed = joints @ rotation.t() + translation[None, :]
        return fisheye_cam.world2camera_pytorch(transformed)
    with torch.no_grad():
        initial_projection = project(identity, initial)
        initial_error = torch.linalg.vector_norm(
45    initial_projection - targets, dim=1
        ).mean()
    initial_z = max(0.1, float(initial[2].item()))
    depth_starts = sorted(set([initial_z, 0.5, 0.75, 1.0, 1.5, 2.0]))
    best = None
50    for depth_start in depth_starts:

```

```

parameter = torch.tensor(
    [
        0.0,
        0.0,
5       0.0,
        float(initial[0]),
        float(initial[1]),
        float(np.log(depth_start)),
    ],
10     device=device,
    dtype=dtype,
    requires_grad=True,
)
optimizer = torch.optim.Adam([parameter], lr=learning_rate)
15 for _ in range(iterations):
    rotation = axis_angle_to_matrix(parameter[:3])
    translation = torch.stack(
        (parameter[3], parameter[4], torch.exp(parameter[5]))
    )
20     projection = project(rotation, translation)
    # Charbonnier-style robust pixel loss. Dividing by 100 keeps the
    # Adam step scale stable while preserving the minimizer.
    residual = (projection - targets) / 100.0
    loss = torch.sqrt(residual.square() + 1e-4).mean()
25     if not torch.isfinite(loss):
        break
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
30     with torch.no_grad():
        rotation_norm = torch.linalg.vector_norm(parameter[:3])
        if rotation_norm > np.pi:
            parameter[:3].mul_(float(np.pi / rotation_norm))
            parameter[3].clamp_(-3.0, 3.0)
35             parameter[4].clamp_(-3.0, 3.0)
            parameter[5].clamp_(float(np.log(0.1)), float(np.log(5.0)))
    with torch.no_grad():
        rotation = axis_angle_to_matrix(parameter[:3])
        translation = torch.stack(
40             (parameter[3], parameter[4], torch.exp(parameter[5]))
        )
        projection = project(rotation, translation)
        error = torch.linalg.vector_norm(projection - targets, dim=1).mean()
        candidate = (
45             float(error),
            rotation.detach().clone(),
            translation.detach().clone(),
            projection.detach().clone(),
        )
50     if best is None or candidate[0] < best[0]:

```

```

        best = candidate
    if best is None:
        raise RuntimeError("Camera translation refinement produced no finite solution")
    return {
5         "rotation": best[1],
        "translation": best[2],
        "initial_projection": initial_projection.detach(),
        "refined_projection": best[3],
        "initial_mean_error_px": float(initial_error),
10        "refined_mean_error_px": float(best[0]),
        "joint_indices": CAMERA_FIT_BODY_INDICES.copy(),
    }
def refine_camera_translation(
15    body_joints_local,
    body_targets_px,
    initial_translation,
    fisheye_cam,
    iterations,
    learning_rate,
20 ):
    """Fit tx, ty, and positive tz while keeping pose and rotation fixed."""
    if iterations <= 0:
        raise ValueError("--camera-fit-iters must be positive")
    if learning_rate <= 0:
25         raise ValueError("--camera-fit-lr must be positive")
    device = body_joints_local.device
    dtype = body_joints_local.dtype
    indices = torch.as_tensor(CAMERA_FIT_BODY_INDICES, device=device, dtype=torch.long)
    joints = body_joints_local.index_select(0, indices).detach()
30    targets = torch.as_tensor(
        body_targets_px[CAMERA_FIT_BODY_INDICES], device=device, dtype=dtype
    )
    initial = initial_translation.detach().to(device=device, dtype=dtype)
    identity = torch.eye(3, device=device, dtype=dtype)
35    def project(translation):
        return fisheye_cam.world2camera_pytorch(joints + translation[None, :])
    with torch.no_grad():
        initial_projection = project(initial)
        initial_error = torch.linalg.vector_norm(
40         initial_projection - targets, dim=1
        ).mean()
    initial_z = max(0.1, float(initial[2].item()))
    depth_starts = sorted(set([initial_z, 0.5, 0.75, 1.0, 1.5, 2.0]))
    best = None
45    for depth_start in depth_starts:
        parameter = torch.tensor(
            [
                float(initial[0]),
                float(initial[1]),
50         float(np.log(depth_start)),

```

```

    ],
    device=device,
    dtype=dtype,
    requires_grad=True,
5   )
    optimizer = torch.optim.Adam([parameter], lr=learning_rate)
    for _ in range(iterations):
        translation = torch.stack(
10         (parameter[0], parameter[1], torch.exp(parameter[2]))
        )
        projection = project(translation)
        residual = (projection - targets) / 100.0
        loss = torch.sqrt(residual.square() + 1e-4).mean()
        if not torch.isfinite(loss):
15         break
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        with torch.no_grad():
20         parameter[0].clamp_(-3.0, 3.0)
        parameter[1].clamp_(-3.0, 3.0)
        parameter[2].clamp_(float(np.log(0.1)), float(np.log(5.0)))
        with torch.no_grad():
            translation = torch.stack(
25             (parameter[0], parameter[1], torch.exp(parameter[2]))
            )
            projection = project(translation)
            error = torch.linalg.vector_norm(projection - targets, dim=1).mean()
            candidate = (
30             float(error),
            translation.detach().clone(),
            projection.detach().clone(),
            )
            if best is None or candidate[0] < best[0]:
35             best = candidate
        if best is None:
            raise RuntimeError("Camera translation refinement produced no finite solution")
        return {
            "rotation": identity,
40         "translation": best[1],
            "initial_projection": initial_projection.detach(),
            "refined_projection": best[2],
            "initial_mean_error_px": float(initial_error),
            "refined_mean_error_px": float(best[0]),
45         "joint_indices": CAMERA_FIT_BODY_INDICES.copy(),
        }
    def draw_camera_fit_debug(
        image_rgb,
        target_body_px,
50     initial_body_px,

```

```

def native_projection(points, focal=25000.):
    return _project(points, focal, [640.,360.])
def constraints(p, source, rigid, cam, lookup):
    v,j=geometry(p); _j0=geometry(source); _jr=geometry(rigid)
5    active=list(MAX_DELTA_DEG); ai=[BODY_POSE_NAMES.index(n) for n in active]
    cap=np.array([MAX_DELTA_DEG[n] for n in active])
    delta=np.rad2deg(np.linalg.norm((p['body_pose']-rigid['body_pose']).reshape(21,3)[ai],axis=1))
    assert np.all(delta<=cap+2e-4)
    assert all(np.array_equal(p[k],source[k]) for k in FIXED)
10    assert np.array_equal(p['body_pose'].reshape(21,3)[[9,10]],source['body_pose'].reshape(21,3)[[9,10]])
    assert np.linalg.norm(p['global_orient']-rigid['global_orient'])<=np.deg2rad(8)+1e-6
    assert .05<=p['transl'].reshape(3)[2]<=3
    floor=np.maximum(np.linalg.norm(j0[[12,13]],axis=1),np.linalg.norm(jr[[12,13]],axis=1))*0.85
    assert np.all(np.linalg.norm(j[[12,13]],axis=1)>=floor-1e-6)
15    assert v[:,2].min()>=.02-1e-6
    names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]['y']]).all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
    idx=[JI[JOINT_MAPPING[n]] for n in names]; target=np.array([[lookup[n]['x'],lookup[n]['y']] for n in names])
20    before=rmse(cam.world2camera(j0.astype(np.float64))[idx],target)
    after=rmse(cam.world2camera(j.astype(np.float64))[idx],target)
    assert after<=before+5.001
    return dict(passed=True,body_delta_deg=dict(zip(active,map(float,delta))),sapiens_body_rmse_px=after
,
25    sapiens_body_source_rmse_px=before,wrist_floor_m=floor.tolist(),body_minimum_z_m=float(
v[:,2].min()))
文件名: egosmplx_pipeline/reference/surface_audit.py
功能: 检查局部非共面三角面的穿插，不作为全身碰撞证明。
"""Local proper noncoplanar crossing checks; not a whole-body collision proof."""
30    import numpy as np
    def segment_triangle(p,q,t):
        d=q-p;e1=t[:,1]-t[:,0];e2=t[:,2]-t[:,0]
        h=np.cross(d,e2);det=np.einsum('ij,ij->i',e1,h)
        valid=np.abs(det)>1e-14;inv=np.zeros_like(det);inv[valid]=1/det[valid]
35    s=p-t[:,0];u=inv*np.einsum('ij,ij->i',s,h);qq=np.cross(s,e1)
        v=inv*np.einsum('ij,ij->i',d,qq);a=inv*np.einsum('ij,ij->i',e2,qq)
        return valid&(u>1e-8)&(v>1e-8)&(u+v<1-1e-8)&(a>1e-8)&(a<1-1e-8)
    def local_intersections(v,f,changed_body_ids):
        """Proper non-coplanar crossings, excluding faces sharing vertices.
40    The test is local: at least one face touches a changed forearm vertex.
        Coplanar overlap/contact is not certified by this test.
        """
        triangles=v[f];lo=triangles.min(1);hi=triangles.max(1)
        active=np.flatnonzero(np.isin(f,changed_body_ids).any(1));active_set=set(active.tolist())
45    pairs=[];tested=0
        for i in active:
            cand=np.flatnonzero((lo<=hi[i]+1e-10).all(1)&(hi>=lo[i]-1e-10).all(1))
            cand=np.array([j for j in cand if (j>i or j not in active_set) and not np.isin(f[j],f[i]).any()],int)
            if len(cand)==0:continue
50    a=np.broadcast_to(triangles[i],(len(cand),3,3));b=triangles[cand];hit=np.zeros(len(cand),bool)

```

```

    for k in range(3):
        hit|=segment_triangle(a[:,k],a[:,(k+1)%3],b)
        hit|=segment_triangle(b[:,k],b[:,(k+1)%3],a)
        tested+=len(cand)
5    pairs.extend([[int(i),int(j)] for j in cand[hit]])
    return dict(proper_crossings=len(pairs),face_pairs=pairs,candidate_face_pairs_tested=tested,
                local_faces_tested=len(active),coplanar_overlaps_checked=False)
文件名: egosmplx_pipeline/reference/wrist_refine.py
功能: 在固定人体形状下优化腕部姿态与投影接缝相容性。
10    """Fixed-shape SMPL-X wrist and projected seam compatibility refinement."""
    from pathlib import Path
    import os
    import sys,copy,argparse,time
    import numpy as np
15    import torch
    import torch.nn.functional as F
    OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'wrist'
    BASE=OUT.parent/'baseline'
    from egosmplx_pipeline.reference.support import (read,write,load,arr,geometry,constraints,PARAMS,FIX
20    ED,SIDES,JI,MAX_DELTA_DEG,BODY_POSE_NAMES,JOINT_MAPPING,smpl_x,FishEyeCameraCa
    librated)
    from egosmplx_pipeline.reference.support import native_projection
    def main():
        ap=argparse.ArgumentParser();ap.add_argument('--steps',type=int,default=500);ap.add_argument('--indice
25    s',type=int,nargs='*');ap.add_argument('--active-projection',action='store_true');a=ap.parse_args()
        torch.backends.cuda.matmul.allow_tf32=False;torch.backends.cudnn.allow_tf32=False
        torch.set_num_threads(2);dev='cuda' if torch.cuda.is_available() else 'cpu';layer=copy.deepcopy(smpl_x.la
        yer['neutral']).to(dev).eval();layer.requires_grad_(False)
        t=lambda x:torch.as_tensor(x, dtype=torch.float32, device=dev)
30    rows=read(BASE/'automatic_input_manifest.json')['frames']
        for ii,row in enumerate(rows):
            if a.indices and ii not in a.indices:continue
            dst=OUT/'active_frames' if a.active_projection else 'frames'/row['condition']/row['camera'];dst.mkdir(pa
            rents=True,exist_ok=True)
35    if (dst/'refinement.json').exists():continue
            bd=BASE/'frames'/row['condition']/row['camera'];p=load(bd/'body_params.npz');source=load(row['source
            _npz']);rigid=load(row['rigid_npz']);w=load(row['wilor_npz']);oldmesh=load(bd/'hybrid_mesh.npz');matche
            s=read(bd/'recipe.json')['matches'];cam=FishEyeCameraCalibrated(row['calibration'])
            lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']}
40    active=list(MAX_DELTA_DEG);ai=torch.tensor([BODY_POSE_NAMES.index(n) for n in active],device
            =dev);caps=t(np.deg2rad([MAX_DELTA_DEG[n] for n in active]))
            b0=t(rigid['body_pose']).reshape(21,3);g0=t(rigid['global_orient']).reshape(3);tr0=t(rigid['transl']).reshape
            (3)
            init_path=OUT/'frames'/row['condition']/row['camera']/'body_params.npz'
45    initp=load(init_path) if a.active_projection and init_path.exists() else p
            delta=(t(initp['body_pose']).reshape(21,3)-b0)[ai].clone().requires_grad_();gd=(t(initp['global_orient']).res
            hape(3)-g0).clone().requires_grad_();tr=t(initp['transl']).reshape(3).clone().requires_grad_()
            names=[n for n in JOINT_MAPPING if lookup[n]['score']>=.3 and np.isfinite([lookup[n]['x'],lookup[n]['y'
            ''])].all() and 0<=lookup[n]['x']<1280 and 0<=lookup[n]['y']<720]
50    bi=torch.tensor([JI[JOINT_MAPPING[n]] for n in names],device=dev);bt=t([[lookup[n]['x'],lookup[n]['y'

```

```

]] for n in names]);bw=t([min(1,lookup[n]['score'])*2 for n in names]);bw/=bw.mean()
ix=torch.tensor([12+SIDES.index(s) for s in matches],device=dev);goals=t([w['joints_2d'][m['candidate'],
0] for s,m in matches.items()])
_,srcj=geometry(source);_,rigj=geometry(rigid);floor=t(np.maximum(np.linalg.norm(srcj[[12,13]],axis=1
5 ),np.linalg.norm(rigj[[12,13]],axis=1))*0.85)
bodylimit=float(np.sqrt(np.mean(np.sum((cam.world2camera(srcj.astype(np.float64))[bi.cpu().numpy()]-
arr(bt))*2,axis=1))))+5
sv,sj=geometry(p);initial_werr=np.linalg.norm(cam.world2camera(sj.astype(np.float64))[ix.cpu().numpy(
)]-arr(goals),axis=1)
10 loopids=[];loopgoals=[]
for s,m in matches.items():
ids=oldmesh[s+' _local_to_mesh'];bv=oldmesh[s+' _wrist_loop_local'];loopids.append(torch.tensor(ids[bv
],device=dev))
uv=native_projection(w['vertices_3d_local'][m['candidate']] + w['cam_t'][m['candidate']]);uv=uv[bv];loop
15 goals.append(t(uv-uv.mean(0)))
def params():
pp={k:t(p[k]).reshape(1,-1) for k in PARAMS};body=b0.clone();body[ai]+=delta;pp.update(body_pose=
body.reshape(1,-1),global_orient=(g0+gd).reshape(1,3));return pp
def forward():
20 o=layer(**params());v=o.vertices[0]+tr;j=o.joints[0,smpl_x.joint_idx]+tr;uv=cam.world2camera_pytorc
h(j);return v,j,uv
def feasible(v,j,uv):
err=(uv[ix]-goals).norm(dim=1)
return bool(v[:,2].min())>=.020001 and (j[[12,13]].norm(dim=1)>=floor+1e-6).all() and torch.sqrt((uv[bi
25 -bt).square().sum(1).mean())<=bodylimit-1e-4 and (err<=(initial_werr)+3).all())
def objective(v,j,uv):
body=(F.smooth_l1_loss((uv[bi]-bt)/t([1280,720]),torch.zeros_like(bt),beta=.003,reduction='none').mean
(1)*bw).mean()
wrists=F.smooth_l1_loss((uv[ix]-goals)/t([1280,720]),torch.zeros_like(goals),beta=.003)
30 seam=[]
for ids,target in zip(loopids,loopgoals):
q=cam.world2camera_pytorch(v[ids]);q=q-q.mean(0);radius=q.square().sum(1).mean().sqrt();rr=target.s
quare().sum(1).mean().sqrt();seam.append(torch.log(radius/rr.clamp_min(1)).square())
shape=torch.stack(seam).mean()
35 priors=.0008*(delta/caps[:,None]).square().mean()+.0002*((tr-tr0)/t([.10,.10,.18])).square().mean()+.000
4*(gd/np.deg2rad(8)).square().mean()
depth=.02*F.relu((.03-v[:,2].min())/0.02).square();guard=.002*F.relu((floor-j[[12,13]].norm(dim=1))/0.05).
square().mean()
return body+4*wrists+.0008*shape+priors+depth+guard
40 opt=torch.optim.Adam([{'params':[delta],'lr':.0015},{'params':[gd],'lr':.0005},{'params':[tr],'lr':.0008}])
with torch.no_grad():v,j,uv=forward();bestloss=float(objective(v,j,uv));best=[x.detach().clone() for x in [d
elta,gd,tr]]
rejected=0;start=time.time()
for step in range(a.steps):
45 opt.zero_grad();v,j,uv=forward();loss=objective(v,j,uv);loss.backward()
previous=[x.detach().clone() for x in [delta,gd,tr]];opt.step()
if a.active_projection:
# Project infeasible Adam proposals back onto nonlinear constraint surfaces.
for project_step in range(12):
50 with torch.no_grad():

```

```

    delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);g
d.mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
    pv,pj,pu=forward()
    br=(pu[bi]-bt).square().sum(1).mean().sqrt()
5    violations=torch.cat([(br-bodylimit+.02).reshape(1)/5,(.02001-pv[:,2].min()).reshape(1)/.02,(floor+1e-5-
pi[[12,13]].norm(dim=1))/.05,((pu[ix]-goals).norm(dim=1)-t(initial_werr)-2.99)/5])
    g=violations.max()
    if float(g)<=0:break
    grads=torch.autograd.grad(g,[delta,gd,tr]);scales=[.0015,.0005,.0008]
10    denom=sum((grad.square().sum()*scale**2) for grad,scale in zip(grads,scales)).clamp_min(1e-12)
    with torch.no_grad():
        for x,grad,scale in zip([delta,gd,tr],grads,scales):x.sub_(1.05*g.detach()*grad*scale**2/denom)
    with torch.no_grad():
        delta.mul_(torch.minimum(torch.ones_like(caps),caps/delta.norm(dim=1).clamp_min(1e-8))[:,None]);gd
15    .mul_(torch.clamp(t(np.deg2rad(8))/gd.norm().clamp_min(1e-8),max=1));tr[2].clamp_(.05,3)
    ok=False
    for trial in range(10):
        v,j,uv=forward()
        if feasible(v,j,uv):ok=True;break
20    for x,prev in zip([delta,gd,tr],previous):x.copy_((x+prev)*.5)
    if not ok:
        rejected+=1
        for x,prev in zip([delta,gd,tr],previous):x.copy_(prev)
        v,j,uv=forward()
25    value=float(objective(v,j,uv))
    if ok and value<bestloss:bestloss=value;best=[x.detach().clone() for x in [delta,gd,tr]]
    if step%100==0:print(row['id'],step,'wrist',arr((uv[ix]-goals).norm(dim=1)).round(2).tolist(),'rejects',rejec
ted,flush=True)
    with torch.no_grad():
30    for x,b in zip([delta,gd,tr],best):x.copy_(b)
    pp=params();saved={k:x.copy() for k,x in p.items()}
    for k,x in pp.items():saved[k]=arr(x).reshape(p[k].shape)
    saved['transl']=arr(tr).reshape(p['transl'].shape)
    candidate={k:x.copy() for k,x in saved.items()};cpu_alpha=1.
35    for trial in range(22):
        for k in ['body_pose','global_orient','transl']:saved[k]=p[k]+cpu_alpha*(candidate[k]-p[k])
        try:acceptance=constraints(saved,source,rigid,cam,lookup);break
        except AssertionError:cpu_alpha*=.9
    else:
40    saved={k:x.copy() for k,x in p.items()};cpu_alpha=0.;acceptance=constraints(saved,source,rigid,cam,loo
kup)
    vv,jj=geometry(saved);local=jj-saved['transl'].reshape(3);pixels=cam.world2camera(jj.astype(np.float64))
    .astype(np.float32)
    saved.update(smplx_mesh_cam=vv,smplx_joint_cam=jj,smplx_joint_img=pixels,smplx_body_joints_3d_
45    local=local[:25],smplx_body_joints_3d_render_cam=jj[:25],smplx_body_joints_2d_render_cam=pixels[:25
],smplx_joint_cam_root_relative=jj-jj[:1])
    if 'smplx_body_joints_3d_cam' in saved:saved['smplx_body_joints_3d_cam']=jj[:25]
    if 'smplx_body_joints_2d' in saved:saved['smplx_body_joints_2d']=pixels[:25]
    for k in FIXED+['left_hand_pose','right_hand_pose']:assert np.array_equal(saved[k],p[k])
50    # Fixed beta kinematic limb lengths (selected joints) are conserved under pose.

```

```

lengths=lambda j:np.array([np.linalg.norm(j[a]-j[b]) for a,b in [(8,10),(10,12),(9,11),(11,13)])]
boneerr=float(np.max(np.abs(lengths(jj)-lengths(sj))));assert boneerr<1e-5
np.savez_compressed(dst/'body_params.npz',**saved)
report=dict(id=row['id'],active_constraint_projection=a.active_projection,steps=a.steps,cpu_feasibility_st
5 ep_fraction=cpu_alpha,elapsed_seconds=time.time()-start>manual_inputs=False,original_hard_bounds_pres
erved=True,body_shape_fixed=True,bone_length_max_change_m=boneerr,wrist_before_px=initial_werr.t
olist(),wrist_after_px=np.linalg.norm(cam.world2camera(jj.astype(np.float64))[ix.cpu().numpy()]-arr(goals
),axis=1).tolist(),matches=matches,loss=bestloss,rejected_steps=rejected,acceptance=acceptance)
write(dst/'refinement.json',report);print('DONE',row['id'],report['wrist_after_px'],flush=True)
10 if __name__=='__main__':main()
文件名： egosmplx_pipeline/reference/wrist_seam.py
功能： 构建有界腕部边界环，并计算接缝两侧的调和位移。
"""Standard-shape bounded wrist loop plus harmonic displacement on both sides."""
from pathlib import Path
15 import os
import sys,argparse
from collections import deque
import numpy as np,cv2
from scipy.sparse import coo_matrix
20 from scipy.sparse.linalg import spsolve
from scipy.spatial.transform import Rotation
OUT=Path(os.environ['EGOSMPLX_REFERENCE_ROOT'])/'wrist';BASE=OUT.parent/'baseline'
from egosmplx_pipeline.reference.support import (read,write,load,geometry,stitch,draw_geometry,FACES
,SIDES,smpl_x,FishEyeCameraCalibrated,write_obj,read_obj,constraints)
25 def adjacency(faces,n):
a=[set() for _ in range(n)]
for face in faces:
for x,y in zip(face,np.roll(face,-1)):a[int(x)].add(int(y));a[int(y)].add(int(x))
return a
30 def distance(a,seeds):
d=np.full(len(a),9999, dtype=int);q=deque()
for v in seeds:d[v]=0;q.append(int(v))
while q:
v=q.popleft()
35 for u in a[v]:
if d[u]>d[v]+1:d[u]=d[v]+1;q.append(u)
return d
def harmonic(a,known,values,free):
"""Dirichlet boundary displacements; everything outside free/known is fixed."""
40 n=len(a);disp=np.zeros((n,3));disp[known]=values;idx=np.full(n,-1,int);idx[free]=np.arange(len(free));rr=[
];cc=[];vv=[];rhs=np.zeros((len(free),3))
for r,v in enumerate(free):
rr.append(r);cc.append(r);vv.append(len(a[v]))
for u in a[v]:
45 if idx[u]>=0:rr.append(r);cc.append(idx[u]);vv.append(-1)
else:rhs[r]+=disp[u]
if len(free):disp[free]=spsolve(coo_matrix((vv,(rr,cc)),shape=(len(free),len(free))).tocsr(),rhs)
return disp
def normals(v,f):return np.cross(v[f[:,1]]-v[f[:,0]],v[f[:,2]]-v[f[:,0]])
50 def edges(f):return np.unique(np.sort(np.concatenate([f[:,[0,1]],f[:,[1,2]],f[:,[2,0]]]),axis=1),axis=0)

```

```

def hand_metrics(v,target,f):
    e=edges(f);rat=np.linalg.norm(v[e[:,0]]-v[e[:,1]],axis=1)/np.maximum(1e-10,np.linalg.norm(target[e[:,0]]-
target[e[:,1]],axis=1))
    n=normals(v,f);nt=normals(target,f)
5    return dict(edge_log_rms=float(np.sqrt(np.mean(np.log(np.maximum(rat,1e-8))*2))),edge_ratio_p01=float
at(np.percentile(rat,1)),edge_ratio_p99=float(np.percentile(rat,99)),normal_reversals_vs_target=int((np.sum
(n*nt,axis=1)<0).sum()),minimum_double_area=float(np.linalg.norm(n,axis=1).min()))
    def build(p,w,matches,cam):
        base,reports=stitch(p,w,matches,cam);original,j=geometry(p);v=base['vertices_cam'].copy();mf=np.asarra
10    y(w['mano_faces']);ha=adjacency(mf,778);report={ }
        neutral={k:x.copy() for k,x in p.items()}
        for k in ['body_pose','global_orient','left_hand_pose','right_hand_pose']:neutral[k]=np.zeros_like(neutral[k
])
        nv,_=geometry(neutral)
15    for s,a in matches.items():
        ids=base[s+'_local_to_mesh'];bv=base[s+'_wrist_loop_local'];global_b=ids[bv];target=base[s+'_lifted_ver
tices'];posed_loop=original[global_b]
        dx=posed_loop-posed_loop.mean(0);dy=target[bv]-target[bv].mean(0)
        u,_vt=np.linalg.svd(dx.T@dy);rotation=u@vt
20    if np.linalg.det(rotation)<0:u[:,-1]*=-1;rotation=u@vt
        rotvec=Rotation.from_matrix(rotation.T).as_rotvec();rm=np.linalg.norm(rotvec);rotvec*=min(1,np.deg2ra
d(30)/max(rm,1e-8));rotation=Rotation.from_rotvec(rotvec).as_matrix().T
        standard_radius=float(np.linalg.norm(nv[global_b]-nv[global_b].mean(0),axis=1).mean());posed_radius=
float(np.linalg.norm(dx,axis=1).mean());target_radius=float(np.linalg.norm(dy,axis=1).mean())
25    scale=np.clip(target_radius/standard_radius,.9,1.1)*standard_radius/posed_radius
        center_move=target[bv].mean(0)-posed_loop.mean(0);center_move*=min(1,.015/max(np.linalg.norm(ce
nter_move),1e-10))
        goal_loop=dx@rotation*scale+posed_loop.mean(0)+center_move
        internal=np.isin(FACES,ids).all(axis=1);bodyfaces=FACES[~internal];ba=adjacency(bodyfaces,len(v));b
30    d=distance(ba,global_b);hd=distance(ha,bv)
        freebody=np.where((bd>0)&(bd<10))[0];freehand=np.where((hd>0)&(hd<7))[0]
        touched_faces=bodyfaces[np.isin(bodyfaces,np.concatenate([global_b,freebody])).any(axis=1)];be=edges
(touched_faces);bn=normals(original,touched_faces);bl=np.linalg.norm(original[be[:,0]]-original[be[:,1]],a
xis=1)
35    before=hand_metrics(v[ids],target,mf);candidates=[]
        for alpha in [0.,.25,.5,.75,1.]:
            newloop=posed_loop+alpha*(goal_loop-posed_loop)
            bdsp=harmonic(ba,global_b,newloop-posed_loop,freebody)
            hdsp=harmonic(ha,bv,newloop-target[bv],freehand)
40    proposal=v.copy();proposal[freebody]=original[freebody]+bdsp[freebody];proposal[ids]=target+hdsp
            ratio=np.linalg.norm(proposal[be[:,0]]-proposal[be[:,1]],axis=1)/bl
            nn=normals(proposal,touched_faces);areas=np.linalg.norm(nn,axis=1)/np.maximum(1e-12,np.linalg.nor
m(bn,axis=1))
            lr=float(np.linalg.norm(proposal[global_b]-proposal[global_b].mean(0),axis=1).mean()/standard_radius)
45    ok=bool(np.isfinite(proposal).all() and proposal[:,2].min()>=.02 and ratio.min()>=.70 and ratio.max()<=1
.40 and areas.min()>=.4 and areas.max()<=2.2 and (np.sum(nn*bn,axis=1)>0).all() and .85<=lr<=1.15)
            hm=hand_metrics(proposal[ids],target,mf)
            if hm['normal_reversals_vs_target']>before['normal_reversals_vs_target']:ok=False
            # Separate geometric acceptance from 2D visual accuracy; no manual input.
50    px=cam.world2camera((smpl_x.orig_hand_regressor[s]@proposal).astype(np.float64));goal=w['joints_2

```

```

d')[a['candidate']];err=float(np.sqrt(np.mean(np.sum((px-goal)**2,axis=1))))
    oldpx=cam.world2camera((smp1_x.orig_hand_regressor[s]@v).astype(np.float64));olderr=float(np.sqrt(n
p.mean(np.sum((oldpx-goal)**2,axis=1))))
    if err>olderr+3:ok=False
5    score=hm['edge_log_rms']+.002*err+.01*hm['normal_reversals_vs_target']
    diag=dict(alpha=alpha,accepted=ok,hand_metrics=hm,hand_rmse_px=err,forearm_edge_ratio_min=float
(ratio.min()),forearm_edge_ratio_max=float(ratio.max()),forearm_area_ratio_min=float(areas.min()),forear
m_area_ratio_max=float(areas.max()),loop_radius_over_standard=lr,score=score)
    candidates.append((diag,proposal))
10    originalpx=cam.world2camera((smp1_x.orig_hand_regressor[s]@v).astype(np.float64));originalerr=float(
np.sqrt(np.mean(np.sum((originalpx-w['joints_2d'][a['candidate']])**2,axis=1))))
    oldscore=before['edge_log_rms']+.002*originalerr+.01*before['normal_reversals_vs_target']
    valid=[c for c in candidates if c[0]['accepted'] and c[0]['score']<oldscore]
    if valid:
15    chosen,v=min(valid,key=lambda c:c[0]['score']);used=True
    else:chosen=dict(alpha=None,hand_metrics=before,hand_rmse_px=originalerr,score=oldscore);used=False
    e
    report[s]=dict(changed=used,standard_wrist_radius_m=standard_radius,posed_wrist_radius_m=posed_ra
dius,lifted_mano_wrist_radius_m=target_radius,baseline_hand_metrics=before,baseline_hand_rmse_px=or
20    iginalerr,selected=chosen,candidates=[d for d,_ in candidates],wrist_displacement_max_m=float(np.linalg.
norm(v[global_b]-original[global_b],axis=1).max()),forearm_support_vertices=int(len(freebody)),hand_tra
nsition_vertices=int(len(freehand)))
    base['vertices_cam']=v.astype(np.float32);base['hand_joints_cam']=np.stack([smp1_x.orig_hand_regressor
[s]@v for s in SIDES]).astype(np.float32)
25    for s in matches:
        base[s+'_legacy_blend_weights']=base.pop(s+'_blend_weights')
        base['method']=np.array('fixed_shape_body_and_bounded_standard_wrist_harmonic_seam')
    return base,report
def main():
30    ap=argparse.ArgumentParser();ap.add_argument('--verify-only',action='store_true');ap.add_argument('--fr
ame-dir',default='frames');a=ap.parse_args();records=[]
    for row in read(BASE/'automatic_input_manifest.json')['frames']:
        d=OUT/a.frame_dir/row['condition']/row['camera']
        if not (d/'refinement.json').exists():continue
35    if not a.verify_only and (d/'geometry_report.json').exists():continue
    p=load(d/'body_params.npz');w=load(row['wilor_npz']);matches=read(d/'refinement.json')['matches'];cam
=FishEyeCameraCalibrated(row['calibration'])
    mesh,report=build(p,w,matches,cam)
    if a.verify_only:
40    stored=load(d/'hybrid_mesh.npz')
    for k in mesh:assert np.array_equal(mesh[k],stored[k]),k
    v,f=read_obj(d/'hybrid_mesh.obj');assert np.array_equal(f,mesh['faces']) and np.max(np.abs(v-mesh['vert
ices_cam']))<1e-7
    lookup={q['name']:q for q in read(row['sapiens_json'])['keypoints']};check=constraints(p,load(row['sourc
e_npz']),load(row['rigid_npz']),cam,lookup)
45    check.update(independent_process_reload=True,mesh_reload_max_abs_m=0.,mesh_minimum_z_m=flo
at(v[:,2].min()),bounded_geometry_passed=True,topology_unchanged_from_seam=True)
    write(d/'acceptance.json',check);print('VERIFIED',row['id'],flush=True);continue
    np.savez_compressed(d/'hybrid_mesh.npz',**mesh);write_obj(d/'hybrid_mesh.obj',mesh['vertices_cam'],
50    mesh['faces'])

```

```

# Keep body-only ablation of the old seam to separate each change.
bm,_=stitch(p,w,matches,cam);np.savez_compressed(d/'body_refined_old_seam.npz',**bm)
image=cv2.imread(row['image']);cv2.imwrite(str(d/'wireframe_skeleton.jpg'),draw_geometry(image,cam,
mesh));cv2.imwrite(str(d/'body_only_wireframe.jpg'),draw_geometry(image,cam,bm))
5   rec=dict(id=row['id'],condition=row['condition'],camera=row['camera'],hands=report);write(d/'geometry_r
eport.json',rec);records.append(rec)
    print('SEAM',row['id'],{s:(x['changed'],x['selected'])['alpha'],round(x['baseline_hand_metrics'] ['edge_log_r
ms'],3),round(x['selected'] ['hand_metrics'] ['edge_log_rms'],3)) for s,x in report.items()},flush=True)
    if not a.verify_only:write(OUT/(a.frame_dir+'_summary.json'),dict(frames=[read(p) for p in sorted((OUT/
10  a.frame_dir).glob('*/*/*geometry_report.json'))],manual_inputs=False,template_shape_and_bone_lengths_fi
xed=True))
    if __name__=='__main__':main()
文件名: tests/test_core.py
功能: 无需模型即可核验数据安全与几何约定的回归检查。
15  """Model-free regression checks for data safety and geometric conventions."""
import json
from pathlib import Path
import tempfile
import unittest
20  import numpy as np
from egosmplx_pipeline.io import load, rmse, write
from egosmplx_pipeline.matching import assign
from egosmplx_pipeline.topology import boundary_edges, graph_distance_from_boundary
from egosmplx_pipeline.lift import native_projection, lift_mano_to_fisheye
25  from egosmplx_pipeline.configuration import load_config, expand
class GeometryTests(unittest.TestCase):
    def test_rmse_is_point_distance(self):
        self.assertEqual(rmse([[3,4]], [[0,0]]), 5)
        self.assertIsNone(rmse(np.empty((0,2)), np.empty((0,2))))
30  def test_ring_distance_and_disconnected_mesh(self):
        faces = np.array([[0,1,4],[1,2,4],[2,3,4],[3,0,4]])
        self.assertEqual(len(boundary_edges(faces)), 4)
        np.testing.assert_array_equal(graph_distance_from_boundary(faces,[0,1,2,3],5), [0,0,0,0,1])
        with self.assertRaises(RuntimeError):
35  graph_distance_from_boundary(faces,[0,1,2,3],6)
    def test_no_duplicate_hand_assignment(self):
        observation = dict(detector_confidence=np.array([.9]), is_right=np.array([0]),
                           joints_2d=np.tile([300.,300.], (1,21,1)))
        lookup = {s+'_wrist':dict(x=300,y=300,score=.9) for s in ['left','right']}
40  matches = assign(observation, lookup, np.zeros((25,2)))
        self.assertEqual(len(matches), 1)
        self.assertEqual(matches['left'] ['candidate'], 0)
    def test_missing_and_low_confidence_hands(self):
        observation = dict(detector_confidence=np.array([.1]), is_right=np.array([1]),
                           joints_2d=np.tile([300.,300.], (1,21,1)))
45  self.assertEqual(assign(observation, {}, np.zeros((25,2))), {})
        observation['detector_confidence'] = np.empty(0)
        self.assertEqual(assign(observation, {}, np.zeros((25,2))), {})
    def test_lift_uses_radial_distance_and_preserves_shifted_projection(self):
50  class Camera:

```

```

def camera2world(self, uv, radial):
    ray = np.c_[uv-[640,360]/1000, np.ones(len(uv))]
    return ray/np.linalg.norm(ray,axis=1)[:,None]*radial[:,None]
points = np.array([[1.,2,1.],[2.,3,1.2]])
5 w = dict(vertices_3d_local=points[None], joints_3d_local=points[None],
    cam_t=np.zeros((1,3)), focal_length_px=np.array(1000.), image_size_wh=np.array([1280,720]))
body_wrist = np.array([2.,3.,7])
result = lift_mano_to_fisheye(Camera(), w, 0, body_wrist, np.array([800,500]))
expected = native_projection(points, 1000, [640,360])
10 expected += [800,500]-expected[0]
actual = native_projection(result['vertices'], 1000, [640,360])
np.testing.assert_allclose(actual, expected, atol=.0001)
self.assertAlmostEqual(float(np.linalg.norm(result['vertices'])[0])), float(np.linalg.norm(body_wrist)),
places=6)
15 class InputTests(unittest.TestCase):
    def test_json_roundtrip_and_nonfinite_rejection(self):
        with tempfile.TemporaryDirectory() as tmp:
            p=Path(tmp)/'x.json';write(p,{'中文':1})
            self.assertEqual(json.loads(p.read_text()), {'中文':1})
20 q=Path(tmp)/'x.npz';np.savez(q,x=np.array([np.nan]))
            with self.assertRaises(ValueError):load(q)
    def test_unresolved_variable_rejected(self):
        with self.assertRaises(ValueError):expand('${EGOSMPLX_MISSING_TEST_VARIABLE}', Path('.'))
25 def test_path_traversal_frame_id_rejected(self):
    with tempfile.TemporaryDirectory() as tmp:
        p=Path(tmp)
        write(p/'frames.json', {'frames':[{'id': '../outside', 'image': 'x.jpg', 'calibration': 'c.json'}]})
        write(p/'config.json', {'paths': {}, 'manifest': 'frames.json', 'output': 'out'})
30 with self.assertRaises(ValueError):load_config(p/'config.json')
if __name__ == '__main__':
    unittest.main()
文件名: tests/test_guided_profile.py
功能: 无需模型即可核验算法配置兼容性与局部表面诊断。
35 """Checks for profile compatibility and honest surface diagnostics without models."""
import json
from pathlib import Path
import tempfile
import unittest
40 import cv2
import numpy as np
from egosmplx_pipeline.configuration import load_config
from egosmplx_pipeline.profiles import CURRENT, LEGACY
from egosmplx_pipeline.reference.surface_audit import local_intersections
45 from egosmplx_pipeline.report import build
class ProfileTests(unittest.TestCase):
    def config(self, directory, **extra):
        p = Path(directory)
        (p/'frames.json').write_text(json.dumps({'frames':[{'id': 'frame_1', 'image': 'i.jpg', 'calibration': 'c.json'}]}))
50 )

```

```

    config = dict(paths={}, manifest='frames.json', output='new_output', **extra)
    (p/'config.json').write_text(json.dumps(config))
    return p/'config.json'

def test_current_default_and_explicit_legacy(self):
5     with tempfile.TemporaryDirectory() as d:
        c, _ = load_config(self.config(d))
        self.assertEqual(c['pipeline_profile'], CURRENT)
        self.assertEqual(c['fitting']['mano_guided_steps'], 1400)
        c, _ = load_config(self.config(d, pipeline_profile=LEGACY))
10     self.assertEqual(c['pipeline_profile'], LEGACY)

def test_unknown_profile_and_invalid_guided_steps_rejected(self):
    with tempfile.TemporaryDirectory() as d:
        with self.assertRaises(ValueError):
            load_config(self.config(d, pipeline_profile='typo'))
15     for value in [0, -1, True, 1.5]:
        with self.assertRaises(ValueError):
            load_config(self.config(d, fitting={'mano_guided_steps':value}))

def test_surface_crossing_and_separated_faces(self):
    vertices=np.array([[0,0,0],[2,0,0],[0,2,0], [.5,.5,-1],[.5,.5,1],[1.5,.5,.5]],float)
20     faces=np.array([[0,1,2],[3,4,5]])
    result=local_intersections(vertices,faces,[0,1,2])
    self.assertEqual(result['proper_crossings'],1)
    self.assertFalse(result['coplanar_overlaps_checked'])
    vertices[3:,0]+=10
25     self.assertEqual(local_intersections(vertices,faces,[0,1,2])['proper_crossings'],0)

def test_report_does_not_turn_reload_pass_into_surface_pass(self):
    with tempfile.TemporaryDirectory() as d:
        root=Path(d)
        for stage in ['raw','body','guided_body','fused']:
30             p=root/'frames/f'/stage;p.mkdir(parents=True)
            cv2.imwrite(str(p/'wireframe.jpg'), np.zeros((72,128,3),np.uint8))
            row=dict(id='f',matched_hands=1,body_error_px=[3,4],hand_error_px=[5],passed=True,
                    pipeline_profile=CURRENT,mano_guided_body=True,local_surface_accepted=False)
            build(root,[row]);summary=json.loads((root/'summary.json').read_text())
35             self.assertTrue(summary['all_reload_checks_passed'])
            self.assertEqual(summary['local_surface_pass_count'],0)
            self.assertEqual(summary['local_surface_audited_count'],1)
            self.assertEqual(cv2.imread(str(root/'f_comparison.jpg')).shape[1],2560)
            row.update(mano_guided_body=False,local_surface_accepted=None,matched_hands=0,hand_error
40             _px=[])
            build(root,[row]);summary=json.loads((root/'summary.json').read_text())
            self.assertIsNone(summary['hand_rmse_px'])
            self.assertEqual(summary['local_surface_audited_count'],0)
            row['pipeline_profile']=LEGACY
45             build(root,[row])
            self.assertEqual(cv2.imread(str(root/'f_comparison.jpg')).shape[1],1920)

if __name__ == '__main__':
    unittest.main()

```

文件名: tests/test_projection.py

50 功能: 核验相机坐标约定与可微投影实现的回归测试。

```

"""Camera convention and differentiable projection regression tests.
Run in body_python. The controller-only environment may omit PyTorch.
"""

import json
5 from pathlib import Path
import tempfile
import unittest
import numpy as np
try:
10     import torch
except ImportError:
    torch = None
@unittest.skipIf(torch is None, 'PyTorch is installed in the configured body worker environment')
class CameraTests(unittest.TestCase):
15     def make_camera(self, **changes):
        from egosmplx_pipeline.camera import FishEyeCameraCalibrated
        calibration = dict(size=[1280,720], polynomialC2W=[-100.], polynomialW2C=[150.,80.,2.],
                           image_center=[630,350], affine=[1.02,.01,-.02],
                           camera_axis_transform=[[0,-1,0],[1,0,0],[0,0,1]])
20     calibration.update(changes)
    with tempfile.TemporaryDirectory() as tmp:
        path=Path(tmp)/'camera.json';path.write_text(json.dumps(calibration))
        return FishEyeCameraCalibrated(path)
    def test_numpy_and_horner_projection_match(self):
25     from egosmplx_pipeline.projection import stable_world2camera
        camera=self.make_camera()
        points=np.array([[.1,.2,.8],[-.2,.1,.4],[.1,-.2,.7]],np.float64)
        tensor=torch.tensor(points,dtype=torch.float64,requires_grad=True)
        pixels=stable_world2camera(camera,tensor)
30     np.testing.assert_allclose(pixels.detach().numpy(),camera.world2camera(points),atol=1e-10)
        pixels.square().sum().backward()
        self.assertTrue(torch.isfinite(tensor.grad).all())
    def test_inverse_depth_is_radial_with_axis_transform(self):
        camera=self.make_camera()
35     pixels=np.array([[600.,400.],[700.,300.]])
        radii=np.array([.4,.8])
        xyz=camera.camera2world(pixels,radii)
        np.testing.assert_allclose(np.linalg.norm(xyz,axis=1),radii,rtol=1e-6)
    def test_singular_affine_rejected(self):
40     with self.assertRaises(ValueError):self.make_camera(affine=[1,1,1])
if __name__ == '__main__':
    unittest.main()
文件名: tools/copyright_layout.py
功能: 按参考版式编排材料, 记录源码来源并导出文档。
45 """Reference typography, source provenance and DOCX/PDF export for EgoSmplx."""
import argparse
import ast
import hashlib
import json
50 import os

```

```

from pathlib import Path
import re
import shutil
import subprocess
5 import sys
import tempfile
from functools import lru_cache
ROOT = Path(__file__).resolve().parents[1]
SOURCE_WIDTH = (11907 - 2 * 1797) / 20
10 SOURCE_SIZE = 9.5
SOURCE_LINES = 50
def digest(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()
def font_setup(font_path=None):
15 """Use reference font metrics without copying proprietary fonts into the repo."""
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    fonts = Path(os.environ.get('WINDIR', 'C:/Windows')) / 'Fonts'
    candidates = [font_path] if font_path else [fonts/'simsun.ttc',
20 Path('/usr/share/fonts/truetype/arphic-gbsn00lp/gbsn00lp.ttf'),
    Path('/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf')]
    chinese = next((p for p in candidates if p and p.is_file()), None)
    if chinese is None:
        raise FileNotFoundError('Provide --font-path with a Chinese-capable TTF/TTC font')
25 pdfmetrics.registerFont(TTFont('EgoChinese', str(chinese), subfontIndex=0))
    latin = fonts/'times.ttf'
    if not latin.is_file():
        latin = Path('/usr/share/fonts/truetype/msttcorefonts/Times_New_Roman.ttf')
    if latin.is_file():
30 pdfmetrics.registerFont(TTFont('EgoLatin', str(latin)))
    @lru_cache(maxsize=4096)
    def character_width(char):
        from reportlab.pdfbase import pdfmetrics
        latin = 'EgoLatin' if 'EgoLatin' in pdfmetrics.getRegisteredFontNames() else 'Times-Roman'
35 return pdfmetrics.stringWidth(char, 'EgoChinese' if ord(char) >= 0x2E80 else latin,
            SOURCE_SIZE)
    def wrap_source(text):
        """Wrap the printed copy only, retaining the repository source indentation."""
        parts, line, width = [], "", 0
40 for char in text.expandtabs(4):
            char_width = character_width(char)
            if line and width + char_width > SOURCE_WIDTH - 4:
                parts.append(line)
                line, width = "", 0
45 line += char
            width += char_width
        return parts + [line]
    def source_listing():
        preferred = ['run_example.sh', 'egosmplx_pipeline/__init__.py',
50 'egosmplx_pipeline/__main__.py', 'egosmplx_pipeline/configuration.py',

```

```

'egosmplx_pipeline/pipeline.py', 'egosmplx_pipeline/frame.py',
'egosmplx_pipeline/bootstrap.py', 'egosmplx_pipeline/io.py',
'egosmplx_pipeline/sampling.py', 'egosmplx_pipeline/egosmplx_predict.py',
'egosmplx_pipeline/sapiens_pose.py', 'egosmplx_pipeline/sapiens_seg.py',
5 'egosmplx_pipeline/wilor_predict.py', 'egosmplx_pipeline/body_common.py',
'egosmplx_pipeline/body_rigid.py', 'egosmplx_pipeline/body_pose.py',
'egosmplx_pipeline/camera.py', 'egosmplx_pipeline/projection.py',
'egosmplx_pipeline/matching.py', 'egosmplx_pipeline/lift.py',
'egosmplx_pipeline/topology.py', 'egosmplx_pipeline/fusion.py',
10 'egosmplx_pipeline/render.py', 'egosmplx_pipeline/report.py']
discovered = sorted(p.relative_to(ROOT).as_posix() for folder in
    ['egosmplx_pipeline', 'tools', 'tests'] for p in (ROOT/folder).rglob('*.*py'))
order = preferred + [p for p in discovered if p not in preferred]
inventory, listing = [], []
15 for number, name in enumerate(order, 1):
    path = ROOT/name
    text = path.read_text(encoding='utf-8')
    lines = text.splitlines()
    entry = dict(file_id='F%02d' % number, path=name, sha256=digest(path),
20     physical_lines=len(lines), nonempty_lines=sum(bool(x.strip()) for x in lines))
    inventory.append(entry)
    description = '命令行运行入口，启动配置检查与完整处理流程。'
    if path.suffix == '.py':
        description = (ast.get_docstring(ast.parse(text)) or '项目程序模块。').splitlines()[0]
25 for label, value in [('文件名: ', name), ('功能: ', description)]:
    for n, part in enumerate(wrap_source(label + value)):
        listing.append(dict(text=part, path=name, source_line=None,
            label=label if n == 0 else ""))
    for source_line, line in enumerate(lines, 1):
30     if line.strip():
        for continuation, part in enumerate(wrap_source(' ' + line)):
            listing.append(dict(text=part, path=name, source_line=source_line,
                continuation=continuation, label=""))
    return inventory, [listing[i:i+SOURCE_LINES] for i in range(0, len(listing), SOURCE_LINES)]
35 def element(parent, tag, **attrs):
    from docx.oxml import OxmlElement
    from docx.oxml.ns import qn
    node = OxmlElement('w:' + tag)
    for key, value in attrs.items():
40     node.set(qn('w:' + key), str(value))
    parent.append(node)
    return node
def set_fonts(run, size=None, east='宋体', latin='Times New Roman', bold=None):
    from docx.oxml.ns import qn
45     from docx.shared import Pt, RGBColor
    run.font.name = latin
    fonts = run._element.get_or_add_rPr().get_or_add_rFonts()
    for key in ['ascii', 'hAnsi', 'cs']:
        fonts.set(qn('w:' + key), latin)
50     fonts.set(qn('w:eastAsia'), east)

```

```

for key in ['asciiTheme', 'hAnsiTheme', 'eastAsiaTheme', 'cstheme']:
    fonts.attrib.pop(qn('w:' + key), None)
if size is not None:
    run.font.size = Pt(size)
5   run.font.color.rgb = RGBColor(0, 0, 0)
    if bold is not None:
        run.bold = bold
def page_field(paragraph, size=11, latin='Cambria'):
    run = paragraph.add_run()
10   set_fonts(run, size, latin=latin)
    element(run._r, 'fldChar', fldCharType='begin')
    element(run._r, 'instrText').text = ' PAGE \\\* MERGEFORMAT '
    element(run._r, 'fldChar', fldCharType='separate')
    element(run._r, 't').text = '1'
15   element(run._r, 'fldChar', fldCharType='end')
def base_document(title, source=False):
    from docx import Document
    from docx.shared import Pt, RGBColor
    from docx.oxml.ns import qn
20   from docx.enum.style import WD_STYLE_TYPE
    document = Document()
    section = document.sections[0]
    if source:
        section.page_width, section.page_height = Pt(11907/20), Pt(16839/20)
25   section.top_margin, section.bottom_margin = Pt(72), Pt(1247/20)
        section.left_margin = section.right_margin = Pt(1797/20)
        section.header_distance, section.footer_distance = Pt(278/20), Pt(0)
        # Only countBy is explicit in the supplied reference document.
        ln = element(section._sectPr, 'lnNumType', countBy=5)
30   section._sectPr.remove(ln)
        section._sectPr.insert_element_before(ln, 'w:pgNumType', 'w:cols', 'w:docGrid')
    else:
        section.page_width, section.page_height = Pt(595.32), Pt(841.92)
        section.top_margin, section.bottom_margin = Pt(72), Pt(56.7)
35   section.left_margin, section.right_margin = Pt(73.7), Pt(68)
        section.header_distance, section.footer_distance = Pt(28.35), Pt(0)
    grid = section._sectPr.find(qn('w:docGrid'))
    if grid is not None:
        grid.set(qn('w:linePitch'), '360')
40   normal = document.styles['Normal']
        normal.font.name = 'Times New Roman'
        normal.font.size = Pt(SOURCE_SIZE if source else 10.5)
        normal._element.get_or_add_rPr().get_or_add_rFonts().set(qn('w:eastAsia'), '宋体')
        fmt = normal.paragraph_format
45   fmt.space_before = fmt.space_after = Pt(0)
        fmt.line_spacing = Pt(14 if source else 20.4)
        fmt.widow_control = not source
        element(normal._element.get_or_add_pPr(), 'snapToGrid', val=0)
    if source:
50   line_style = document.styles.add_style('line number', WD_STYLE_TYPE.CHARACTER, builtin=Tr

```

```

ue)
    line_style._element.attrib.pop(qn('w:customStyle'), None)
    line_style.base_style = document.styles['Default Paragraph Font']
    line_style.font.name = 'Cambria'
5    line_style.font.size = Pt(11)
else:
    fmt.first_line_indent = Pt(21)
    for name, size in [('Title', 22), ('Heading 1', 14), ('Heading 2', 12), ('Heading 3', 11)]:
        style = document.styles[name]
10    for border in list(style._element.xpath('./w:pPr/w:pBdr')):
        border.getparent().remove(border)
        style.font.name, style.font.size, style.font.bold = 'Times New Roman', Pt(size), True
        style.font.color.rgb = RGBColor(0, 0, 0)
        style._element.get_or_add_rPr().get_or_add_rFonts().set(qn('w:eastAsia'), '黑体')
15    fmt = style.paragraph_format
    fmt.first_line_indent = Pt(0)
    fmt.space_before, fmt.space_after = Pt(12 if name == 'Heading 1' else 8), Pt(4)
    fmt.line_spacing, fmt.keep_with_next = Pt(20.4), True
    for name in ['Caption', 'TOC 1', 'TOC 2', 'TOC 3']:
20    style = (document.styles[name] if name in document.styles else
              document.styles.add_style(name, WD_STYLE_TYPE.PARAGRAPH))
    style.font.name, style.font.size = 'Times New Roman', Pt(10.5)
    style._element.get_or_add_rPr().get_or_add_rFonts().set(qn('w:eastAsia'), '宋体')
    style.paragraph_format.first_line_indent = Pt(0)
25    style.paragraph_format.line_spacing = Pt(18)
header_style = document.styles['Header']
header_style.font.name = 'Cambria' if source else 'Times New Roman'
header_style.font.size = Pt(11 if source else 9)
header_style.paragraph_format.space_before = header_style.paragraph_format.space_after = Pt(0)
30 header_style.paragraph_format.line_spacing = 1
header_style.paragraph_format.first_line_indent = Pt(0)
header = section.header.paragraphs[0]
if source:
    # Two empty header paragraphs are part of the supplied reference layout.
35    section.header.add_paragraph(style='Header')
    header = section.header.add_paragraph(style='Header')
header.alignment = 2
header.paragraph_format.first_line_indent = Pt(0)
header.paragraph_format.line_spacing = 1
40 header.paragraph_format.space_after = Pt(0)
set_fonts(header.add_run(title + (' ' * 18 if source else ' ' * 8)), 9,
           latin='宋体' if source else 'Times New Roman')
if not source:
    set_fonts(header.add_run('第 '), 9)
45 page_field(header, 11 if source else 9, 'Cambria' if source else 'Times New Roman')
if source:
    element(element(header._p.get_or_add_pPr(), 'pBdr'), 'bottom',
            val='single', sz=4, space=1, color='auto')
else:
50    set_fonts(header.add_run(' 页'), 9)

```

```

        section.different_first_page_header_footer = True
        first = section.first_page_header.paragraphs[0]
        first.alignment = 2
        set_fonts(first.add_run('第 '), 9)
5        page_field(first, 9, 'Times New Roman')
        set_fonts(first.add_run(' 页'), 9)
        section.footer.paragraphs[0].clear()
        document.core_properties.title = title + (' 源程序文档' if source else ' 软件设计说明书')
        document.core_properties.author = 'EgoSmplx'
10        document.core_properties.subject = '软件著作权登记鉴别材料'
        return document

def source_docx(path, title, pages):
    from docx.shared import Pt
    document = base_document(title, source=True)
15    for page_index, page in enumerate(pages):
        for line_index, row in enumerate(page):
            paragraph = document.add_paragraph()
            fmt = paragraph.paragraph_format
            if page_index and line_index == 0:
20                fmt.page_break_before = True
            fmt.keep_together = fmt.keep_with_next = False
            fmt.line_spacing = Pt(14)
            if row['label']:
                label = row['label']
25                set_fonts(paragraph.add_run(label), SOURCE_SIZE, bold=True)
                set_fonts(paragraph.add_run(row['text'][len(label):]), SOURCE_SIZE)
            else:
                set_fonts(paragraph.add_run(row['text']), SOURCE_SIZE)
        document.save(path)
30    def inline(paragraph, text, size=10.5, bold=False):
        text = re.sub(r'\\([\\]]+\\)\\([\\^]+\\)', r'^1', text)
        for token in re.split(r'(^|\\*|\\.|\\?|\\*|\\`|\\^|\\+)', text):
            if not token:
                continue
35            strong = token.startswith('*') and token.endswith('*')
            code = token.startswith('') and token.endswith('')
            value = token[2:-2] if strong else token[1:-1] if code else token
            if code and len(value) > 24:
                value = re.sub(r'([/_])', lambda m: m[0] + '\\u200b', value)
40            set_fonts(paragraph.add_run(value), size, bold=bold or strong)

def add_table(document, rows):
    from docx.shared import Pt
    from docx.enum.table import WD_TABLE_ALIGNMENT, WD_CELL_VERTICAL_ALIGNMENT
    rows = [r for r in rows if not all(re.fullmatch(r'[:\\s]+', x or '-') for x in r)]
45    if not rows:
        return
    count = max(len(row) for row in rows)
    table = document.add_table(rows=0, cols=count)
    table.alignment, table.autofit = WD_TABLE_ALIGNMENT.CENTER, False
50    weights = [max(6, min(34, max(len(r[c]) if c < len(r) else 0 for r in rows)))

```

```

        for c in range(count)]
    if rows[0] == ['条件', '处理', '输出']:
        weights = [35, 35, 30]
        widths = [(595.32 - 73.7 - 68) * v / sum(weights) for v in weights]
5    for col, width in zip(table.columns, widths):
        col.width = Pt(width)
    borders = element(table._tbl.tblPr, 'tblBorders')
    for side in ['top', 'left', 'bottom', 'right', 'insideH', 'insideV']:
        element(borders, side, val='single', sz=4, color='D9D9D9')
10    margins = element(table._tbl.tblPr, 'tblCellMar')
    for side, value in [('top', 75), ('bottom', 75), ('left', 80), ('right', 80)]:
        element(margins, side, w=value, type='dxa')
    for index, row in enumerate(rows):
        cells = table.add_row().cells
15    element(table.rows[-1]._tr.get_or_add_trPr(), 'cantSplit')
    if index == 0:
        element(table.rows[-1]._tr.get_or_add_trPr(), 'tblHeader')
    for col, cell in enumerate(cells):
        cell.width = Pt(widths[col])
20    cell.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
    text = row[col] if col < len(row) else ""
    paragraph = cell.paragraphs[0]
    fmt = paragraph.paragraph_format
    fmt.first_line_indent, fmt.line_spacing = Pt(0), Pt(13)
25    fmt.space_before = fmt.space_after = Pt(0)
    paragraph.alignment = 1 if len(text) <= 8 else 0
    inline(paragraph, text.replace('<br>', '\n'), size=9, bold=index == 0)
    element(cell._tc.get_or_add_tcPr(), 'shd',
            fill='E7EDF4' if index == 0 else 'F7F9FB' if index % 2 == 0 else 'FFFFFF')
30    after = document.add_paragraph()
    after.paragraph_format.first_line_indent = Pt(0)
    after.paragraph_format.line_spacing = Pt(5)
    after.paragraph_format.space_after = Pt(3)
def image_block(document, caption, target, markdown_path):
35    from docx.shared import Pt
    from PIL import Image
    path = (markdown_path.parent/target).resolve()
    if not path.is_file():
        raise FileNotFoundError('Missing document illustration: ' + str(path))
40    with Image.open(path) as source:
        width, height = source.size
    render_width = min(595.32 - 73.7 - 68, 465 * width / height)
    paragraph = document.add_paragraph()
    paragraph.alignment = 1
45    fmt = paragraph.paragraph_format
    fmt.first_line_indent = Pt(0)
    fmt.space_before, fmt.space_after, fmt.line_spacing = Pt(6), Pt(4), 1
    fmt.keep_with_next = True
    picture = paragraph.add_run().add_picture(str(path), width=Pt(render_width))
50    picture._inline.docPr.set('descr', caption)

```

```

label = document.add_paragraph(style='Caption')
label.alignment = 1
label.paragraph_format.keep_together = True
label.paragraph_format.space_after = Pt(7)
5 inline(label, caption, 10.5)
def design_docx(path, title, markdown_path, cover=True):
    from docx.shared import Pt
    document = base_document(title)
    if cover:
10     paragraph = document.add_paragraph(style='Title')
        paragraph.alignment = 1
        fmt = paragraph.paragraph_format
        fmt.space_before, fmt.space_after, fmt.line_spacing = Pt(120), Pt(30), Pt(35)
        name = title.replace('第一视角下视双目鱼眼相机', '第一视角下视双目鱼眼相机\n', 1)
15     set_fonts(paragraph.add_run(name), 22, east='黑体', bold=True)
        paragraph = document.add_paragraph()
        paragraph.alignment = 1
        paragraph.paragraph_format.first_line_indent = Pt(0)
        set_fonts(paragraph.add_run('软件设计说明书'), 20, east='黑体', bold=True)
20     paragraph = document.add_paragraph()
        paragraph.alignment = 1
        paragraph.paragraph_format.first_line_indent = Pt(0)
        paragraph.paragraph_format.space_before = Pt(22)
        set_fonts(paragraph.add_run('概要设计与详细设计'), 12)
25     document.add_page_break()
        paragraph = document.add_paragraph('目录', style='Title')
        paragraph.alignment = 1
        paragraph.paragraph_format.space_after = Pt(12)
        run = document.add_paragraph().add_run()
30     element(run._r, 'fldChar', fldCharType='begin')
        element(run._r, 'instrText').text = ' TOC \o "1-2" \h \z \u '
        element(run._r, 'fldChar', fldCharType='separate')
        element(run._r, 't').text = '更新目录'
        element(run._r, 'fldChar', fldCharType='end')
35     document.add_page_break()
    lines = markdown_path.read_text(encoding='utf-8').splitlines()
    index = 0
    first_main_heading = True
    while index < len(lines):
40         line = lines[index].strip()
        index += 1
        if not line or line == '---' or line.startswith('# ') or line == '软件设计说明书':
            continue
        heading = re.match(r'^(\#{2,4})s+(.+)', line)
45     if heading:
        level = len(heading[1]) - 1
        paragraph = document.add_paragraph(style='Heading ' + str(level))
        if level == 1:
            paragraph.paragraph_format.page_break_before = not first_main_heading and not heading[2].sta
50 rtswith('第三部分')

```

```

        first_main_heading = False
        inline(paragraph, heading[2], size={1:14, 2:12, 3:11}[level], bold=True)
        for run in paragraph.runs:
            set_fonts(run, east='黑体', bold=True)
5         continue
    if line.startswith('|'):
        rows = [[v.strip() for v in line.strip('|').split('|')]]
        while index < len(lines) and lines[index].strip().startswith('|'):
            rows.append([v.strip() for v in lines[index].strip().strip('|').split('|')])
10         index += 1
        add_table(document, rows)
        continue
    matched = re.fullmatch(r'!\[([^\]]*)\]\([([^\)]+)\)', line)
    if matched:
15         image_block(document, matched[1], matched[2], markdown_path)
        continue
    if line.startswith('```'):
        language, block = line[3:].strip(), []
        while index < len(lines) and not lines[index].strip().startswith('```'):
20         block.append(lines[index])
            index += 1
        index += 1
        if language == 'mermaid':
            continue
25         for value in block:
            paragraph = document.add_paragraph()
            fmt = paragraph.paragraph_format
            fmt.first_line_indent, fmt.line_spacing, fmt.space_after = Pt(0), Pt(12), Pt(0)
            set_fonts(paragraph.add_run(value), 8.5, latin='Consolas')
30         document.add_paragraph().paragraph_format.line_spacing = Pt(5)
        continue
    paragraph = document.add_paragraph()
    paragraph.alignment = 3
    bullet = re.match(r'^[-*]\s+(.+)', line)
35     if bullet:
        paragraph.paragraph_format.first_line_indent = Pt(0)
        paragraph.paragraph_format.left_indent = Pt(12)
        inline(paragraph, '• ' + bullet[1])
    else:
40         inline(paragraph, line)
    element(document.settings._element, 'updateFields', val='true')
    document.save(path)
def export_pdfs(paths, backend='auto'):
    if backend == 'auto':
45         backend = 'word' if sys.platform == 'win32' else 'libreoffice'
    if backend == 'word':
        if sys.platform != 'win32':
            raise RuntimeError('Word PDF export requires Windows and installed Microsoft Word')
        script = r"param([string]$InputJson)
50     $ErrorActionPreference = 'Stop'

```

```

$taskWord = $null
try {
    $taskWord = New-Object -ComObject Word.Application
    $taskWord.Visible = $false
5    $taskWord.DisplayAlerts = 0
    $taskWord.AutomationSecurity = 3
    foreach ($taskPath in (Get-Content -LiteralPath $InputJson -Raw -Encoding UTF8 | ConvertFrom-Json))
    {
        $taskDocument = $null
10    try {
            $taskDocument = $taskWord.Documents.Open([string]$taskPath, $false, $false)
            $taskDocument.Repaginate()
            $taskDocument.Fields.Update() | Out-Null
            foreach ($taskToc in $taskDocument.TablesOfContents) { $taskToc.Update() }
15    $taskDocument.Repaginate()
            foreach ($taskToc in $taskDocument.TablesOfContents) { $taskToc.UpdatePageNumbers() }
            $taskDocument.Save()
            $taskPdf = [System.IO.Path]::ChangeExtension([string]$taskPath, '.pdf')
            $taskDocument.ExportAsFixedFormat($taskPdf, 17)
20    Write-Output ([System.IO.Path]::GetFileName($taskPdf))
        } finally { if ($null -ne $taskDocument) { $taskDocument.Close(0) } }
    }
} finally { if ($null -ne $taskWord) { $taskWord.Quit() } }
""

25    with tempfile.TemporaryDirectory(prefix='egosmplx_doc_export_') as temporary:
        script_path, inputs = Path(temporary)/'export.ps1', Path(temporary)/'inputs.json'
        script_path.write_text(script, encoding='utf-8-sig')
        inputs.write_text(json.dumps([str(p.resolve()) for p in paths], ensure_ascii=False), encoding='utf-8-
sig')
30    subprocess.run(['powershell.exe', '-NoProfile', '-NonInteractive', '-ExecutionPolicy',
        'Bypass', '-File', str(script_path), str(inputs)], check=True)
    else:
        executable = shutil.which('soffice') or shutil.which('libreoffice')
        if not executable:
35    raise FileNotFoundError('LibreOffice is required for PDF export; --docx-only retains editable source')
        with tempfile.TemporaryDirectory(prefix='egosmplx_lo_') as temporary:
            for path in paths:
                subprocess.run([executable, '-env:UserInstallation=' + Path(temporary).as_uri(),
40                '--headless', '--convert-to', 'pdf', '--outdir', str(path.parent), str(path)], check=True)
            for path in paths:
                if not path.with_suffix('.pdf').is_file():
                    raise RuntimeError('PDF export did not produce ' + str(path.with_suffix('.pdf')))
def main():
45    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--output', type=Path, default=ROOT/'copyright/generated')
    parser.add_argument('--font-path', type=Path)
    parser.add_argument('--only', choices=['requested', 'all'], default='requested')
    parser.add_argument('--pdf-backend', choices=['auto', 'word', 'libreoffice'], default='auto')
50    parser.add_argument('--docx-only', action='store_true')

```

```

args = parser.parse_args()
font_setup(args.font_path)
args.output.mkdir(parents=True, exist_ok=True)
metadata = json.loads((ROOT/'software.json').read_text(encoding='utf-8'))
5 title = metadata['name_zh'] + ' ' + metadata['registration_version']
inventory, full_pages = source_listing()
selected = (list(range(len(full_pages))) if len(full_pages) <= 60 else
            list(range(30)) + list(range(len(full_pages)-30, len(full_pages))))
outputs = []
10 for suffix, pages in [('完整', full_pages), ('交存', [full_pages[i] for i in selected])]:
    path = args.output/'03_源程序文档_' + suffix + '.docx'
    source_docx(path, title, pages)
    outputs.append(path)
design = args.output/'02_软件设计说明书.docx'
15 design_docx(design, title, ROOT/'docs/DESIGN_zh.md')
outputs.append(design)
if args.only == 'all':
    for name in ['01_登记信息填表稿', '04_权属核对与提交清单']:
        path = args.output/(name+'.docx')
20 design_docx(path, title, ROOT/'copyright'/(name+'.md'), cover=False)
        outputs.append(path)
if not args.docx_only:
    export_pdfs(outputs, args.pdf_backend)
page_map = [[dict(printed_line=n+1, **{key: row.get(key) for key in
25 ['path', 'source_line', 'continuation']}) for n, row in enumerate(page)]
              for page in full_pages]
index = dict(software=title, files=inventory,
             physical_source_lines=sum(x['physical_lines'] for x in inventory),
             nonempty_source_lines=sum(x['nonempty_lines'] for x in inventory),
30 printed_source_lines=sum(len(p) for p in full_pages), full_source_pages=len(full_pages),
             deposited_original_pages=[i+1 for i in selected], source_lines_per_page=SOURCE_LINES,
             reference_layout=dict(page_twips=[11907,16839], margins_twips=dict(top=1440,right=1797,
                                     bottom=1247,left=1797,header=278,footer=0), font_latin='Times New Roman',
                                     font_east_asia='宋体',font_points=9.5,line_spacing_points=14,line_number_every=5),
35 listing_rule='Actual nonempty source plus file/function headings; wrap printed copy only; no filler',
             source_page_map=page_map, ownership_status='Applicant must verify rights and third-party bounda-
ries')
    (args.output/'source_inventory.json').write_text(json.dumps(index,indent=2,ensure_ascii=False)+'\n', enc-
oding='utf-8')
40 (ROOT/'docs/SOURCE_INDEX.md').write_text('# 源码索引\n\n' + '\n'.join(
    '- %s: %s (%d 行, 非空 %d 行)' % (x['file_id'],x['path'],x['physical_lines'],x['nonempty_lines'])
    for x in inventory) + '\n', encoding='utf-8')
    print(json.dumps({k:index[k] for k in ['physical_source_lines','nonempty_source_lines',
    'printed_source_lines','full_source_pages']},ensure_ascii=False))
45 if __name__ == '__main__':
    main()

```

文件名: tools/draw_design_figures.py

功能: 依据项目验证记录绘制可复现的设计说明书图表。

"""Draw reproducible, code-native figures for the EgoSmplx design document.

50 The numeric charts use the checked-in current validation evidence. The raster

figures are drawn directly with Pillow at 2x resolution; matching editable SVGs are generated from the same drawing commands. No inference result is fabricated or modified by this script.

"""

```

5  from __future__ import annotations
    import argparse
    import html
    import json
    import math
10  import os
    import re
    from pathlib import Path
    from PIL import Image, ImageDraw, ImageFont
    ROOT = Path(__file__).resolve().parents[1]
15  ASSETS = ROOT / "docs" / "assets"
    WIDTH, HEIGHT, SCALE = 1800, 1120, 2
    COLORS = {
        "navy": "#14304D", "teal": "#087F8C", "orange": "#D97528",
        "blue": "#3767A1", "text": "#213B50", "muted": "#5D7284",
20  "line": "#D2DEE8", "bg": "#F4F8FB", "white": "#FFFFFF",
        "teal_bg": "#E8F6F4", "orange_bg": "#FFF2E6", "blue_bg": "#EAF1FB",
        "red": "#B54D45", "red_bg": "#FAECE9", "green": "#0E8270",
    }
    FONT_FILES: dict[str, Path] = {}
25  def configure_fonts(font_dir: str | None = None):
        """Find existing Chinese fonts; never download or install font resources."""
        supplied = font_dir or os.environ.get("EGOSMPLX_FONT_DIR")
        directories = []
        if supplied:
30  directory = Path(supplied).expanduser()
            if not directory.is_dir():
                raise ValueError(f"Font directory does not exist: {directory}")
            directories.append(directory)
        directories.extend([
35  Path("C:/Windows/Fonts"),
        Path("/usr/share/fonts/truetype/microsoft"),
        Path("/usr/share/fonts/truetype/msttcorefonts"),
        Path("/usr/share/fonts/opentype/noto"),
        Path("/usr/share/fonts/truetype/noto"),
40  Path("/usr/share/fonts/truetype/wqy"),
        Path("/usr/share/fonts/opentype/source-han-sans"),
        Path("/usr/share/fonts/truetype/dejavu"),
        Path("/usr/share/fonts/truetype/liberation2"),
        Path("/usr/local/share/fonts"),
45  Path.home() / ".local/share/fonts",
        Path.home() / ".fonts",
    ])
    candidates = {
        "regular": ["msyh.ttc", "msyh.ttf", "simsun.ttc", "simhei.ttf",
50  "NotoSansCJK-Regular.ttc", "NotoSansCJKsc-Regular.otf",

```

```

        "NotoSansSC-Regular.ttf", "NotoSerifCJK-Regular.ttc",
        "SourceHanSansSC-Regular.otf", "wqy-microhei.ttc",
        "wqy-zenhei.ttc"],
    "bold": ["msyhbd.ttf", "msyhbd.ttf", "simhei.ttf",
5         "NotoSansCJK-Bold.ttc", "NotoSansCJKsc-Bold.otf",
        "NotoSansSC-Bold.ttf", "NotoSerifCJK-Bold.ttc",
        "SourceHanSansSC-Bold.otf"],
    "mono": ["consola.ttf", "DejaVuSansMono.ttf",
10         "LiberationMono-Regular.ttf", "NotoSansMonoCJK-Regular.ttc"],
    }
    # Search each directory in order so --font-dir takes precedence.
    discovered = []
    for directory in dict.fromkeys(directories):
        if directory.is_dir():
15         discovered.append({p.name.lower(): p for p in directory.rglob("*")
                            if p.is_file() and p.suffix.lower() in {".ttf", ".ttc", ".otf"}}})
    def find(names):
        for files in discovered:
            for name in names:
20             if name.lower() in files:
                return files[name.lower()]
        return None
    regular = find(candidates["regular"])
    if regular is None:
25         raise ValueError(
            "No supported Chinese font was found. Provide an existing font directory "
            "with --font-dir PATH or EGOSMPLX_FONT_DIR. Supported fonts include "
            "Microsoft YaHei (msyh.ttc), SimSun (simsun.ttc), Noto Sans/Serif CJK, "
            "Source Han Sans SC, and WenQuanYi. No fonts are downloaded automatically.")
30     FONT_FILES.update(regular=regular, bold=find(candidates["bold"]) or regular,
                       mono=find(candidates["mono"]) or regular)
    class Figure:
        def __init__(self, name: str, title: str, subtitle: str):
            self.name = name
35             self.im = Image.new("RGB", (WIDTH * SCALE, HEIGHT * SCALE), COLORS["bg"])
            self.draw = ImageDraw.Draw(self.im)
            self.svg = [f<svg xmlns="http://www.w3.org/2000/svg" width="{WIDTH}" height="{HEIGHT}" vie
wBox="0 0 {WIDTH} {HEIGHT}">',
                        f<rect width="{WIDTH}" height="{HEIGHT}" fill="{COLORS["bg"]}"/>']
40             self.rect(0, 0, WIDTH, 148, "navy", radius=0)
            self.rect(54, 34, 9, 74, "teal", radius=4)
            self.text(86, 35, title, 48, "white", bold=True)
            self.text(87, 99, subtitle, 26, "#D5E5F3")
            @staticmethod
45             def color(value: str):
                return COLORS.get(value, value)
            def font(self, size: int, bold=False, mono=False):
                if not FONT_FILES:
                    configure_fonts()
50                 file = FONT_FILES["mono" if mono else "bold" if bold else "regular"]

```

```

    return ImageFont.truetype(str(file), round(size * SCALE))
def text(self, x, y, content, size=32, fill="text", bold=False, anchor="left", mono=False, max_width=None):
    content = str(content)
    font = self.font(size, bold, mono)
    if max_width:
        while self.draw.textlength(content, font=font) > max_width * SCALE and size > 18:
            size -= 1
            font = self.font(size, bold, mono)
    tw = self.draw.textlength(content, font=font) / SCALE
    xx = x - tw / 2 if anchor == "center" else x - tw if anchor == "right" else x
    self.draw.text((round(xx * SCALE), round(y * SCALE)), content, font=font, fill=self.color(fill), anchor="left")
    family = "Consolas, monospace" if mono else "Microsoft YaHei, sans-serif"
    weight = "700" if bold else "400"
    self.svg.append(f'<text x="{xx:.2f}" y="{y}" dominant-baseline="text-before-edge" font-family="{family}" font-size="{size}" font-weight="{weight}" fill="{self.color(fill)}">{html.escape(content)}</text>')
    return tw
def lines(self, x, y, lines, size=30, fill="text", gap=12, **kwargs):
    for i, line in enumerate(lines):
        self.text(x, y + i * (size + gap), line, size, fill, **kwargs)
def rect(self, x, y, w, h, fill="white", stroke=None, radius=18, line_width=2):
    box = (round(x*SCALE), round(y*SCALE), round((x+w)*SCALE), round((y+h)*SCALE))
    self.draw.rounded_rectangle(box, radius=round(radius*SCALE), fill=self.color(fill), outline=self.color(stroke) if stroke else None, width=round(line_width*SCALE))
    self.svg.append(f'<rect x="{x}" y="{y}" width="{w}" height="{h}" rx="{radius}" fill="{self.color(fill)}" stroke="{self.color(stroke)}" stroke-width="{line_width}" if stroke else "">')
def line(self, points, fill="muted", width=3, dashed=False):
    coords = [(round(x*SCALE), round(y*SCALE)) for x, y in points]
    if dashed:
        for a, b in zip(coords[:-1], coords[1:]):
            d=math.dist(a,b)
            for start in range(0, int(d), 18*SCALE):
                end=min(start+10*SCALE,d)
                aa=(a[0]+(b[0]-a[0])*start/d,a[1]+(b[1]-a[1])*start/d)
                bb=(a[0]+(b[0]-a[0])*end/d,a[1]+(b[1]-a[1])*end/d)
                self.draw.line([aa,bb],fill=self.color(fill),width=width*SCALE)
    else:
        self.draw.line(coords, fill=self.color(fill), width=width*SCALE, joint="curve")
    self.svg.append('<polyline points="'+'.join(f'{x},{y}' for x,y in points)+f'" fill="none" stroke="{self.color(fill)}" stroke-width="{width}" stroke-linejoin="round"'+(' stroke-dasharray="10 8"' if dashed else "")+>')
def arrow(self, points, fill="teal", width=4):
    self.line(points,fill,width)
    a,b=points[-2:]
    angle=math.atan2(b[1]-a[1],b[0]-a[0]); head=14
    ps=[b,(b[0]-head*math.cos(angle-.5),b[1]-head*math.sin(angle-.5)),(b[0]-head*math.cos(angle+.5),b[1]-head*math.sin(angle+.5))]
    self.draw.polygon([(round(x*SCALE),round(y*SCALE)) for x,y in ps],fill=self.color(fill))
    self.svg.append('<polygon points="'+'.join(f'{x:.2f},{y:.2f}' for x,y in ps)+f'" fill="{self.color(fill)}"/>')

```

```

>')
def circle(self, x, y, r, fill="teal", stroke=None, width=2):
    self.draw.ellipse((round((x-r)*SCALE),round((y-r)*SCALE),round((x+r)*SCALE),round((y+r)*SCALE)),fill=self.color(fill),outline=self.color(stroke) if stroke else None,width=width*SCALE)
5     self.svg.append(f'<circle cx="{x}" cy="{y}" r="{r}" fill="{self.color(fill)}"'+(f' stroke="{self.color(stroke)}" stroke-width="{width}"' if stroke else "")+'>')
def card(self,x,y,w,h,title,lines,accent="teal", details_size=29, title_size=35):
    self.rect(x,y,w,h,"white", "line")
    self.rect(x,y,8,h,accent,radius=4)
10    self.text(x+28,y+24,title,title_size,"navy",bold=True,max_width=w-54)
    self.lines(x+28,y+83,lines,details_size,"muted",gap=13,max_width=w-54)
def label(self,x,y,w,text,fill="teal",bg="teal_bg",size=27):
    self.rect(x,y,w,50,bg,radius=25)
    self.text(x+w/2,y+8,text,size,fill,bold=True,anchor="center",max_width=w-20)
15 def footer(self,source):
    self.line([(55,1040),(1745,1040)],"line",2)
    self.text(56,1058,source,24,"muted",max_width=1430)
    self.text(1744,1058,"EgoSmplx V1.0",24,"navy",anchor="right")
def save(self):
20    ASSETS.mkdir(parents=True,exist_ok=True)
    self.im.save(ASSETS/(self.name+".png"),dpi=(300,300),optimize=True)
    (ASSETS/(self.name+".svg")).write_text("\n".join(self.svg+['</svg>']),encoding='utf-8')
    print(self.name)
def architecture():
25     f=Figure("design_architecture","模块架构与运行边界","配置驱动的逐帧流水线·多模型独立环境·可核验的结果资产")
    f.rect(55,188,1690,115,"blue_bg",radius=18)
    f.text(86,214,"统一控制层",34,"navy",bold=True)
    f.text(374,219,"CLI / 配置加载 / 输入预检 / SHA-256 指纹 / 日志 / 校验后续跑",32,"navy")
30    f.arrow([(900,303),(900,348)])
    f.card(55,358,360,382,"输入与观测",[f"图像、每相机鱼眼标定","EgoSMPLX 初始参数","Sapiens2 关节点与分割","WiLoR / MANO 手部"],"blue")
    f.card(490,358,405,382,"身体与手部校正",[f"刚体 → 19 关节微调","左右手一对一匹配","有界腕点校正","可见前臂轮廓选择"],"teal")
35    f.card(970,358,355,382,"增强与融合",[f"MANO 辅助身体优化","固定形状和骨长","原有有界轮廓接缝","局部谐波位移"],"teal")
    f.card(1400,358,345,382,"验收与导出",[f"独立重载 / OBJ 核验","拓扑 / 硬约束诊断","局部表面穿插检查","报告、网格与结果包"],"orange")
    for a,b in [(415,490),(895,970),(1325,1400)]:f.arrow([(a,548),(b-9,548)])
40    f.rect(55,790,1690,184,"navy",radius=18)
    f.text(86,818,"运行边界",34,"white",bold=True)
    f.text(325,823,"cam3、cam4 逐张处理，各用对应标定；当前不进行双目联合三角化或时序跟踪。",30,"#E3EFF8")
    f.text(86,890,"外部依赖",31,"white",bold=True)
45    f.text(325,893,"EgoSMPLX / Sapiens2 / WiLoR 模型环境、权重与 SMPL-X / MANO 模型资源",30,"#E3EFF8")
    f.footer("依据：README.md、CURRENT_PIPELINE_zh.md 与 pipeline.py。")
    f.save()
def pipeline():
50     f=Figure("design_pipeline","默认处理流程与四阶段结果","MANO 辅助身体优化 → 原有有界轮廓

```

接缝融合")

```

xs=[55,500,945,1390];w=355
top=[("01","输入预检",["帧清单、图像和标定","相机尺寸与摘要核对"]),
      ("02","自动观测",["EgoSMPLX / Sapiens2","WiLoR 手部候选"]),
      ("03","身体两阶段拟合",["全局姿态与平移","19 关节及刚体微调"]),
      ("04","腕点与轮廓校正",["冻结一对一身份","有限候选、可见轮廓"])]
5  bot=[("08","报告与结果包",["四列投影与误差汇总","资产摘要、诊断与 ZIP"]),
      ("07","独立验收",["参数 / 网格 / OBJ 重载","局部穿插单独记录"]),
      ("06","原接缝融合",["鱼眼反投影放置 MANO","局部谐波与形变限制"]),
10  ("05","MANO 辅助身体",["身体、腕点、掌根、腕环","冻结轮廓与姿态先验"])]
for x,(num,title,lines) in zip(xs,top):
    f.card(x,210,w,238,title,lines,"blue",28,33)
    f.circle(x+30,192,27,"blue");f.text(x+30,173,num,27,"white",bold=True,anchor="center")
for x,(num,title,lines) in zip(xs,bot):
15  f.card(x,562,w,238,title,lines,"orange" if num in ["07","08"] else "teal",28,33)
    f.circle(x+30,544,27,"orange" if num in ["07","08"] else "teal");f.text(x+30,525,num,27,"white",bold=
True,anchor="center")
    for a,b in [(410,500),(855,945),(1300,1390)]:f.arrow([(a,330),(b-10,330)],"blue")
    f.arrow([(1567,448),(1567,519)],"teal")
20  for a,b in [(1390,1300),(945,855),(500,410)]:f.arrow([(a,683),(b+10,683)],"teal")
    f.rect(55,866,1690,112,"white","line")
    f.text(85,892,"四列展示",31,"navy",bold=True)
    labels=[("raw","原始预测"),("body","Sapiens2 身体"),("guided_body","MANO 辅助身体"),("fused","
最终融合")]
25  for i,(name,label) in enumerate(labels):
        x=300+i*350
        f.text(x,882,label,28,"navy",bold=True)
        f.text(x,930,name+" / ",25,"muted",mono=True)
    f.footer("依据: CURRENT_PIPELINES_zh.md; 无匹配手时保留原身体, 跳过新增手部增强。")
30  f.save()
def data_contract():
    f=Figure("design_data_contract","数据接口与结果组织","原始观测 → 标准身体参数 → 混合格网及
    独立诊断")
    f.card(55,204,490,238,"输入清单与观测",["manifest: id / image / calibration","原图像素坐标、逐帧
35  图像摘要","NPZ 参数 / JSON 关节点 / NPY 分割"],"blue",27)
    f.card(640,204,495,238,"标准身体阶段",["raw / body / guided_body","params.npz: SMPL-X 参数及
    派生量","mesh.obj / wireframe.jpg"],"teal",27)
    f.card(1230,204,515,238,"混合格网与报告",["mesh.npz / 同几何 OBJ","recipe、冻结目标和验收 JSO
    N","index.html / summary.json / ZIP"],"orange",27)
40  f.arrow([(545,323),(630,323)])
    f.arrow([(1135,323),(1220,323)])
    f.rect(55,500,1690,478,"white","line")
    f.text(88,527,"最终融合网格的重建依赖",36,"navy",bold=True)
    xleft,xright=88,650
45  rows=[("body_params.npz","最终身体基底参数"),
          ("wilor_predictions.npz","匹配的 MANO 手部预测"),
          ("recipe.json + calibration","冻结身份、配方与相机标定"),
          ("silhouette_targets.json","冻结轮廓目标与顶点支持集"),
          ("segmentation_labels.npy","图像部位标签")]
50  for i,(name,desc) in enumerate(rows):

```

```

y=593+i*66
f.circle(102,y+17,6,"teal")
f.text(126,y,name,28,"navy",mono=True)
f.text(xright,y,desc,29,"muted")
5 f.rect(1240,586,455,305,"teal_bg",radius=18)
f.text(1269,612,"融合结果",33,"teal",bold=True)
f.lines(1269,675,["由身体基底与手部、标定、","冻结目标共同恢复。","单一标准 SMPL-X 参数","
无法恢复最终混合网格。"],28,"navy",gap=16)
f.footer("依据: DATA_FORMAT_zh.md; 三维坐标单位为米, 二维误差使用原始图像像素。")
10 f.save()
def guided_constraints():
    f=Figure("design_guided_constraints","MANO 辅助身体模块: IPO 与约束","在已有身体上连续优化·原接缝算法单独执行")
    f.card(55,205,440,343,"输入 I",["腕点 / 轮廓校正后的身体","Sapiens2 自动身体点","WiLoR 腕点和
15 非拇指 MCP","MANO 16 点腕环与冻结轮廓"],"blue",28)
    f.card(630,205,540,343,"处理 P",["优化 19 个身体关节及刚体","Huber 残差+姿态先验","Adam 最
多 1400 步","约束投影、迭代回退、CPU 复核"],"teal",28)
    f.card(1305,205,440,343,"输出 O",["guided_body 标准参数","网格 / 关节 / 四阶段投影","optimization.json 日志","无可行新增参数则保留原身体"],"orange",28)
20 f.arrow([(495,381),(620,381)])
f.arrow([(1170,381),(1295,381)])
f.rect(55,591,1690,96,"navy",radius=18)
f.text(84,620,"组合目标",31,"white",bold=True)
f.text(309,618,"4×身体 + 4×手腕 + 掌根 + 5×腕环 + 1.5×轮廓 + 先验",34,"white")
25 cards=[("保持固定",["betas、手指、面部、双脚","骨长、匹配身份、轮廓支持集"]),
("几何硬约束",["整体角增量 ≤ 8°","平移 z: 0.05–3 m; 网格 z ≥ 0.02 m"]),
("误差与回退",["身体 RMSE 设上界","连续 200 步无有效改善即停止"])]
for x,(title,lines) in zip([55,630,1205],cards):f.card(x,741,540,227,title,lines,"teal",27,32)
f.footer("依据: CURRENT_PIPELINE_zh.md、reference/guided_body.py; 自动拟合不读取人工标
30 注。")
f.save()
def metrics():
    data=json.loads((ROOT/'docs/current_body_rmse.json').read_text(encoding='utf-8'))
    # Hand values are explicitly published in the current eight-frame table.
35 text=(ROOT/'docs/CURRENT_PIPELINE_zh.md').read_text(encoding='utf-8')
hand=re.search(r'WiLoR 手部 294 点 RMSE \ ( [ \d . ] + ) px \ ( [ \d . ] + ) px',text)
if not hand:raise ValueError('Current hand metric row was not found')
rows=[("身体: 100 点",data['pooled_rmse_px']['before_guided'],data['pooled_rmse_px']['latest']),
("身体: 排除鼻点 94 点",data['excluding_nose_rmse_px']['before_guided'],data['excluding_nose_r
40 mse_px']['latest']),
("肩 / 肘 / 腕: 47 点",data['upper_limb_rmse_px']['before_guided'],data['upper_limb_rmse_px']['lat
est']),
("手部: 294 点",float(hand[1]),float(hand[2]))]
f=Figure("design_metrics","四场景八帧: 自动目标点拟合误差","同批观测、14 只匹配手·合并所
45 有点平方误差计算 RMSE·越低越好")
f.label(55,183,326,"增强前自动基线","blue","blue_bg")
f.label(410,183,402,"增强身体+原接缝","teal","teal_bg")
f.text(1744,194,"单位: px",29,"muted",anchor="right")
f.text(1715,250,"变化",26,"muted",anchor="right")
50 plotx,plotw=535,1050

```

```

for tick in range(0,61,10):
    x=plotx+plotw*tick/60
    f.line([(x,274),(x,854)],"line",2)
    f.text(x,868,str(tick),27,"muted",anchor="center")
5  for i,(label,before,after) in enumerate(rows):
    y=284+i*142
    f.text(55,y+22,label,32,"navy",bold=True,max_width=435)
    f.rect(plotx,y,plotw*before/60,35,"blue",radius=9)
    f.rect(plotx,y+49,plotw*after/60,35,"teal",radius=9)
10  f.text(plotx+plotw*before/60+16,y-2,f'{before:.2f}',29,"blue",bold=True)
    f.text(plotx+plotw*after/60+16,y+47,f'{after:.2f}',29,"teal",bold=True)
    delta=after-before
    f.text(1715,y+20,('+' if delta>0 else '-') + f'{delta:.2f}',29,"orange" if delta>0 else "teal",bold=True,anch
or="right")
15  f.rect(55,937,1690,72,"orange_bg",radius=18)
    f.text(86,957,"身体指标下降，手部指标上升；这些指标以 Sapiens2 / WiLoR 自动点为参考，不代表三维精度。",29,"navy")
    f.footer("数据：current_body_rmse.json；手部 294 点见 CURRENT_PIPELINE_zh.md / VALIDATIO
N_zh.md。")
20  f.save()
    def validation():
        data=json.loads((ROOT/'docs/guided_migration_validation.json').read_text(encoding='utf-8'))
        frames=data['frames'];n=len(frames)
        reload_count=sum(bool(r['independent_reload']) for r in frames)
        surface_count=sum(bool(r['local_crossings_passed']) for r in frames)
25  f=Figure("design_validation","八帧验收：重载与局部穿插分别报告","独立重建及 OBJ 一致性通过
，不自动代表表面视觉精度合格")
    f.rect(55,190,817,193,"teal_bg",radius=20)
    f.text(86,220,"独立重载通过",33,"teal",bold=True)
30  f.text(810,229,f'{reload_count}/{n}',78,"teal",bold=True,anchor="right")
    f.text(86,303,"全部网格字段逐值一致，最大顶点差 0 m",27,"navy")
    f.rect(928,190,817,193,"orange_bg",radius=20)
    f.text(959,220,"局部穿插检查通过",33,"orange",bold=True)
    f.text(1683,229,f'{surface_count}/{n}',78,"orange",bold=True,anchor="right")
35  f.text(959,303,"固定前臂 / 腕口支持集，对完整表面检查",27,"navy")
    f.rect(55,432,1690,462,"white","line")
    headers=[('四场景',86),('cam3 重载',665),('cam3 局部',917),('cam4 重载',1199),('cam4 局部',1475)]
    for title,x in headers:f.text(x,458,title,29,"navy",bold=True)
    f.line([(85,517),(1715,517)],"line",2)
40  conditions=[('condition_01','常规姿态'),('condition_02','手物交互'),('condition_03','手物遮挡'),('conditi
on_04','失败病例')]
    for i,(key,label) in enumerate(conditions):
        y=548+i*79
        if i%2==0:f.rect(74,y-7,1651,65,"bg",radius=8)
45  f.text(86,y,label,30,"navy",bold=True)
        f.text(333,y+3,key,25,"muted",mono=True)
        for cam,x1,x2 in [('cam3',726,981),('cam4',1260,1539)]:
            row=next(r for r in frames if r['condition'].startswith(key) and r['camera']==cam)
            for x,passed in [(x1,row['independent_reload']), (x2,row['local_crossings_passed'])]:
50  f.label(x-73,y-1,146,"通过" if passed else "未通过","teal" if passed else "red","teal_bg" if passed

```

```

else "red_bg",26)
    f.rect(55,932,1690,78,"navy",radius=18)
    f.text(87,952,"验收范围：不检查共面重叠；不构成全身无碰撞证明；失败帧保留诊断和实际输出
。",30,"white")
5    f.footer("数据：guided_migration_validation.json；优化重跑 2 帧，网络与八帧上游身体拟合未全部
重跑。")
    f.save()
if __name__=='__main__':
    parser = argparse.ArgumentParser(description=__doc__)
10    parser.add_argument("--font-dir", help="Existing font directory; overrides EGOSMPLX_FONT_DIR")
    args = parser.parse_args()
    try:
        configure_fonts(args.font_dir)
    except ValueError as error:
15        parser.error(str(error))
    for draw_figure in [architecture,pipeline,data_contract,guided_constraints,metrics,validation]:
        draw_figure()
文件名： tools/export_copyright.py
功能： 按参考版式导出实际源码和图文设计说明书。
20 #!/usr/bin/env python3
    """Export actual source and illustrated design documents in the reference layout."""
    from copyright_layout import main
    if __name__ == '__main__':
        main()
25 文件名： tools/verify_guided_migration.py
功能： 使用已保存的自动观测输入，核验增强人体与原始接缝实现。
    """Verify the migrated guided-body/original-seam implementation on saved automatic inputs.
    Requires a frozen automatic manifest and the 2026-09-29 expected experiment.
    Selected frames rerun the full added optimizer; all frames rebuild and reload
30    final meshes. Neural networks and upstream two-stage fitting are not rerun.
    """

    import argparse
    import os
    from pathlib import Path
35    import shutil
    import sys
    import numpy as np
    sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
    from egosmplx_pipeline.io import read, write, load, sha
40    def main():
        ap = argparse.ArgumentParser(description=__doc__)
        ap.add_argument('--manifest', type=Path, required=True)
        ap.add_argument('--expected', type=Path, required=True)
        ap.add_argument('--output', type=Path, required=True)
45        ap.add_argument('--repo-root', type=Path, required=True)
        ap.add_argument('--config', type=Path, required=True)
        ap.add_argument('--optimize-indices', type=int, nargs='*', default=[0, 6])
        args = ap.parse_args()
        if args.output.exists():
50            raise FileExistsError(args.output)

```

```

args.output.mkdir(parents=True)
os.environ['EGOSMPLX_REPO_ROOT'] = str(args.repo_root.resolve())
os.environ['EGOSMPLX_CONFIG'] = str(args.config.resolve())
os.environ['EGOSMPLX_REFERENCE_ROOT'] = str(args.output.resolve())
5 from egosmplx_pipeline.reference import guided_body as guided
from egosmplx_pipeline.reference.guided_fuse import run as fuse
rows = read(args.manifest)['frames']
records = []
for i, row in enumerate(rows):
10     expected = args.expected / 'frames' / row['condition'] / row['camera']
    destination = guided.frame_dir(row)
    destination.mkdir(parents=True, exist_ok=True)
    # context() verifies the manifest's per-input SHA-256 before using observations.
    guided.context(row)
15     if i in args.optimize_indices:
        guided.fit(row, 1400)
    else:
        shutil.copyfile(expected / 'body_params.npz', destination / 'body_params.npz')
        actual_p, expected_p = load(destination / 'body_params.npz'), load(expected / 'body_params.npz')
20     core = guided.PARAMS + ['transl']
    exact = all(np.array_equal(actual_p[k], expected_p[k]) for k in core)
    if not exact:
        raise AssertionError('Body optimization differs from expected frame ' + row['id'])
    fuse(row)
25     fuse(row, verify=True)
    actual, baseline = load(destination / 'hybrid_mesh.npz'), load(expected / 'hybrid_mesh.npz')
    equality = {k: bool(np.array_equal(actual[k], baseline[k])) for k in baseline if k in actual}
    if set(actual) != set(baseline) or not all(equality.values()):
        raise AssertionError('Fusion differs from expected frame ' + row['id'])
30     record = dict(id=row['id'], condition=row['condition'], camera=row['camera'],
        optimizer_rerun=i in args.optimize_indices, body_parameters_bitwise_equal=exact,
        all_mesh_fields_bitwise_equal=True,
        vertex_max_abs_m=float(np.abs(actual['vertices_cam']-baseline['vertices_cam']).max()),
        local_crossings_passed=read(destination / 'geometry_validation.json')['all_local_crossing_che
35     cks_passed'],
        independent_reload=read(destination / 'reload_validation.json')['passed'])
    records.append(record)
    write(args.output / 'progress.json', dict(frames=records))
    print('MIGRATION_VERIFIED', row['id'], flush=True)
40     write(args.output / 'migration_validation.json', dict(frame_count=len(records),
        manifest_sha256=sha(args.manifest), optimizer_rerun_count=len(args.optimize_indices),
        all_body_parameters_bitwise_equal=all(r['body_parameters_bitwise_equal'] for r in records),
        all_mesh_fields_bitwise_equal=True, local_surface_pass_count=sum(r['local_crossings_passed'] for
r in records),
45     scope='Added optimizer rerun on selected frozen automatic inputs; all final meshes independently re
built and reloaded; networks and upstream fitting not rerun', frames=records))
if __name__ == '__main__':
    main()

```